

Data Base Management System

Contents

1	Introduction	3		
1.1	Information	3		
1.2	Database	3		
2	Introduction to Database Systems	3		
2.1	Motivation and Everyday Examples	3		
2.1.0.1	Contacts (Phonebook/Contacts App).	3		
2.1.0.2	Banking.	4		
2.1.0.3	E-commerce.	4		
2.1.0.4	Hospital records.	4		
2.1.0.5	From files/spreadsheets to a DBMS.	4		
2.1.0.6	1) Capture.	4		
2.1.0.7	2) Store.	4		
2.1.0.8	3) Process.	4		
2.1.0.9	4) Share.	4		
2.1.0.10	5) Secure.	4		
2.1.0.11	Running mini-case: College Library.	4		
2.1.0.12	Key takeaways.	4		
2.2	Data, Information, and Knowledge	4		
2.2.0.1	Note:	5		
3	File System Approach (Before DBMS)	6		
3.1	What Are File Systems?	6		
3.2	Drawbacks of File Systems	6		
3.3	Why DBMS Is Better Than File Systems	6		
3.3.0.1	File System vs DBMS (Shop Example)	6		
4	DBMS (Database Management System)	7		
4.1	Components of DBMS Applications	7		
	Hardware • Software • Data • Procedures • Database Access Language • People • (Memorable)			
4.2	Advantages of Database Management System (DBMS)	7		
4.3	Types of DBMS	11		
5	Data Models	11		
5.1	Introduction to Data Model	11		
5.2	Hierarchical Database	11		
5.2.0.1	Where It Is Best Used	11		
5.2.0.2	Example of Hierarchical Database	11		
5.3	Network Database	12		
5.3.0.1	Structure of a Network Database	12		
5.3.0.2	Comparison with Hierarchical Model	12		
5.3.0.3	Why Network Database Is Useful	12		
5.3.0.4	Applications of Network Database	12		
5.3.0.5	Examples of Network Databases	12		
5.3.0.6	Limitation (for Students to Remember)	12		
5.3.0.7	Relation DBMS :	13		
5.3.0.8	Object-Oriented DBMS :	13		
5.4	Cloud-Based Database	13		
5.4.0.1	Popular Cloud Platforms	14		
5.4.0.2	How It Works	14		
5.4.0.3	Key Features of Cloud-Based Databases	14		
5.4.0.4	Types of Cloud Databases	14		
5.4.0.5	Where Cloud Databases Are Used	15		
5.4.0.6	Examples of Cloud-Based Databases	15		
5.5	NoSQL DBMS	15		
5.5.0.1	Data Models in NoSQL	15		
5.5.0.2	Key-Value Store	15		
5.5.0.3	Document Store	15		
5.5.0.4	Column-Based Store	15		
5.5.0.5	Graph Database	15		
5.5.0.6	Why NoSQL is Powerful	15		
5.5.0.7	Types of Data Handled	15		
5.5.0.8	Popular NoSQL Databases	15		
5.5.0.9	Where NoSQL is Used	15		
5.6	Advantages and Disadvantages of DBMS	15		
5.6.0.1	Advantages of DBMS	15		
5.6.0.2	Disadvantages of DBMS	16		
6	DBMS Architecture	17		
6.1	Single-Tier Architecture	17		
	Single-Tier Architecture (1-Tier)			
6.2	Two-Tier Architecture	17		
	Two-Tier Architecture (2-Tier)			
6.3	Three-Tier Architecture	17		
	Three-Tier Architecture (3-Tier)			
7	Data Independence	18		
7.1	Data Independence	18		
7.2	Logical Data Independence	18		
7.3	Physical Data Independence	18		
8	Database Languages	19		
8.1	Data Definition Language (DDL)	19		
8.2	Data Manipulation Language (DML)	19		
8.3	Data Definition Language (DDL) Commands	19		
	1. CREATE • 2. ALTER • 3. TRUNCATE • 4. DROP			
8.4	Applications of Data Definition Language (DDL)	20		
8.5	Data Control Language (DCL)	20		
	DCL Commands • GRANT Command • REVOKE Command			
8.6	Transaction Control Language (TCL)	20		
	COMMIT Command • ROLLBACK Command • SAVEPOINT Command • Working of TCL Commands			
8.7	Real-Life Example of Data Independence	21		
	Logical Data Independence • Physical Data Independence			
9	Database Users	22		
9.1	Database Administrator (DBA)	22		
9.2	Different Types of Database Users	22		
	Database Administrator (DBA)			
9.3	Developers	22		
9.4	End Users	22		
10	Database Design Phases	23		
10.1	Requirement Collection & Analysis	23		
10.2	Conceptual Design Phase	23		
10.3	Logical Design Phase	23		
10.4	Physical Design Phase	23		
11	Introduction to E-R Model	24		
11.1	Entity Type, Entity Set and Attributes	24		
	Entity • Attributes • Entity Set:			
11.2	Attribute :	25		
	Types of Attributes			
11.2.1.1	Relationship :	25		
11.3	Relationship Type and Relationship Set	26		
11.4	Key Constraint :	26		
11.5	Participation Constraint :	27		
	1. One-to-One (1:1) • 2. One-to-Many (1:M) • 3. Many-to-One (M:1) • 4. Many-to-Many (M:N)			
11.6	Generalization, Specialization and Aggregation in ER Model	29		
11.6.0.1	Generalization :	29		

11.6.0.2	Specialization :	30	18.5	 5. E-Commerce Websites	50
11.6.0.3	Aggregation :	30	18.6	 6. Payroll System	50
12	Relational Data Model	33			
12.1	Relational Schema :	33			
12.2	Tables, Attributes, and Tuples	33			
12.3	Types of Keys in DBMS	33			
12.4	What is a Foreign Key?	33			
12.5	Candidate Key	33			
12.6	Composite Key	33			
12.7	Unique Key	34			
12.8	Natural Key	34			
13	Introduction of Network Model	34			
13.1	Key Points	34			
13.2	Advantages of Network Model in DBMS	35			
13.3	Disadvantages of Network Model in DBMS	35			
14	Object-Oriented Data Model	36			
14.1	Elements of Object-Oriented Data Model	36			
15	Relational Database Design	37			
15.1	The Relational Model	37			
15.1.0.1	Intoduction To The Relational Model	37			
15.2	RDBMS: Collection of Relations	37			
15.2.0.1	Keys	37			
15.2.0.2	Primary Key (PK)	38			
15.2.0.3	Alternate Key (AK)	38			
15.2.0.4	Foreign Key (FK) and Relationships	39			
16	Normalization	40			
16.1	Benefits of Database Normalization	40			
16.2	Need of Normalization	40			
	<i>Insertion Anomaly • Update Anomaly • Deletion Anomaly</i>				
16.3	First Normal Form (1NF)	40			
	<i>Example • Example: Transforming a Table into First Normal Form (1NF) • SQL Example: 1NF Transformation • Example: First Normal Form (1NF) • SQL Example: First Normal Form (1NF) • Real-World Drawbacks of First Normal Form (1NF)</i>				
16.4	Second Normal Form (2NF)	44			
16.5	Third Normal Form (3NF)	45			
16.6	Boyce-Codd Normal Form (BCNF)	46			
16.7	Higher Normal Forms (4NF, 5NF, and Domain-Key Normal Form)	47			
16.8	Decomposition and Dependency Preservation	47			
17	Transactions and Concurrency Control	48			
17.1	Transaction Concepts	48			
	<i>ACID Properties</i>				
17.2	States of a Transaction	48			
17.3	Serializability	48			
	<i>Conflict Serializability • View Serializability</i>				
17.4	Concurrency Control Techniques	48			
	<i>Locks</i>				
17.4.1.1	Shared Locks	48			
17.4.1.2	Exclusive Locks	48			
	<i>Two-Phase Locking Protocol • Timestamp Ordering Protocol</i>				
17.5	Deadlock	48			
	<i>Deadlock Detection • Deadlock Prevention • Deadlock Recovery</i>				
18	Applications of DBMS	49			
18.1	 1. Banking System	49			
18.2	 2. Railway and Airline Reservation Systems	49			
18.3	 3. Hospital Management System	49			
18.4	 4. University and School Systems	50			

1. Introduction

Data refers to raw, unprocessed facts and figures. These facts can be in the form of numbers, names, text, images, audio, or videos. By themselves, data does not have any meaning. It only becomes meaningful when it is processed and organized.

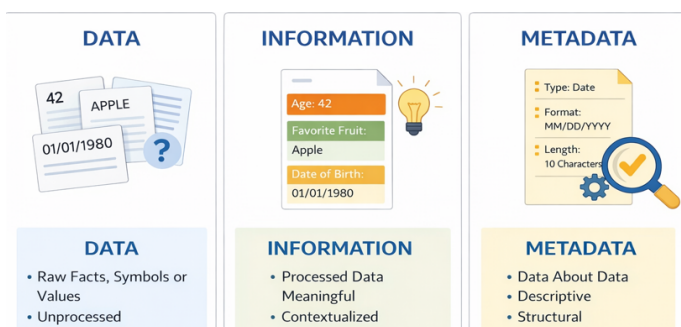
Example: Marks = 85, 90, 78 → These are just numbers. This is data.

Data alone is meaningless unless it is processed.

1.1. Information

Information is processed and organized data that has meaning. When raw data is arranged in a proper way and given context, it becomes useful and understandable.

Example: Data: 25, Information: The age of Suresh is 25

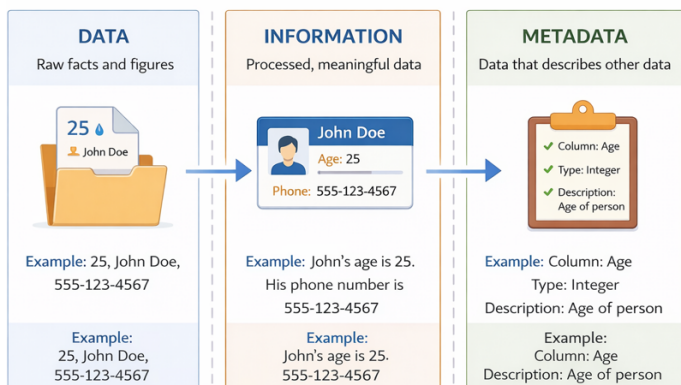


Metadata Metadata means data about data. It describes the structure and properties of actual data stored in a database. Metadata tells us how the data is organized, what type of data it is, and what rules apply to it.

It includes information like column names, data types, size, constraints, and storage format.

Example: If a table has a column named Age of type Integer with a size of 2 digits, this information is called metadata.

Data vs Information vs Metadata

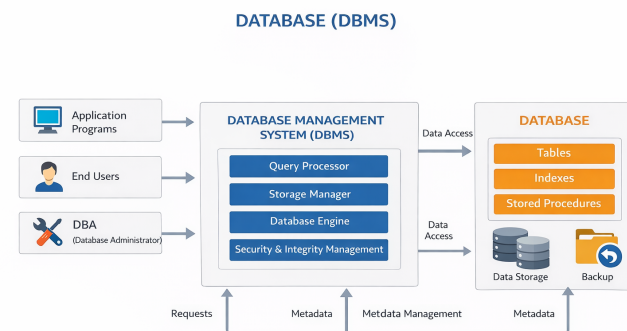
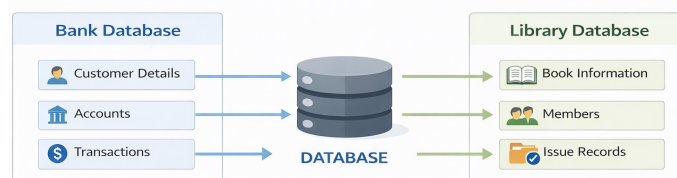


1.2. Database

A database is an organized collection of related data stored in a structured way so that it can be easily accessed, managed, and updated. Only related data forms a database; random or unrelated data does not.

Examples:

- A bank database stores customer details, accounts, and transactions.
- A library database stores book information, members, and issue records.



Why is Data Important?
because:

- It helps in decision making
- It helps businesses grow
- It helps understand customer behavior
- It helps predict future trends

Example:

Amazon collects reviews (data) → processes them → finds if a product is good or bad (information) → improves business decisions.

2. Introduction to Database Systems

2.1. Motivation and Everyday Examples

Databases are everywhere in daily life, often hidden behind the apps and websites we use. Any place where data must be *kept accurately*, *found quickly*, and *updated safely* benefits from a Database Management System (DBMS). This subsection builds intuition using familiar scenarios and then connects them to a simple, repeatable data lifecycle: capture → store → process → share → secure.

Why we need databases: contacts, banking, e-commerce, hospital records

Contacts (Phonebook/Contacts App).

- Data kept:** Name, phone numbers, emails, birthdays, tags (e.g., family, work).

- **Typical actions:** Add a new contact, update a number, search by name, merge duplicates.
- **Why a DBMS helps:** Avoids duplicate entries, supports fast search (indexes), keeps consistent updates across devices via sync, and maintains backups.

Banking.

- **Data kept:** Customer profile, accounts, balances, transactions, cards, logins.
- **Typical actions:** Money transfer, ATM withdrawal, bill payment, statement generation.
- **Why a DBMS helps:** Ensures *accuracy* and *consistency* (your balance must always be correct), supports *concurrent* access (many users at once), enforces *security* and *audit trails*, and guarantees *recovery* from failures.

E-commerce.

- **Data kept:** Products, prices, stock, carts, orders, payments, deliveries, reviews.
- **Typical actions:** Search products, add to cart, place order, track shipment, write a review.
- **Why a DBMS helps:** Keeps inventory consistent while many customers shop simultaneously, enables powerful search and recommendations, and supports order tracking reliably.

Hospital records.

- **Data kept:** Patients, visits, diagnoses, lab tests, prescriptions, billing, staff schedules.
- **Typical actions:** Register a patient, schedule an appointment, record test results, generate bills, share discharge summaries.
- **Why a DBMS helps:** Protects sensitive health data (privacy), links related information (patient ↔ visits ↔ lab results), reduces duplicate tests, and improves care coordination.

From files/spreadsheets to a DBMS.

- **Problems without a DBMS:** Repeated data (redundancy), mismatched values (inconsistency), accidental overwrites, slow search in large files, no central control for security/backup.
- **What a DBMS adds:** Centralized storage, rules (constraints) to keep data valid, a query language for fast retrieval, transaction support for safe multi-user updates, and built-in backup/recovery.

Data lifecycle: *capture* → *store* → *process* → *share* → *secure*

The same simple pipeline appears across domains. Below, we explain each stage with a short, concrete example.

1) Capture.

- **What it is:** Collecting raw facts at the source (forms, sensors, user input, imports).
- **Examples:** A user signs up (name, email, password); a cashier scans a product barcode; a lab device produces a test result.
- **Goal:** Get accurate, complete input with basic checks (e.g., email format, required fields).

2) Store.

- **What it is:** Saving captured data in structured tables/collections with meaningful fields and types.
- **Examples:** Insert a new Customer row; update Inventory quantity; record a Transaction.
- **Goal:** Organize data so it can be found and updated efficiently (use keys, indexes, constraints).

3) Process.

- **What it is:** Turning stored data into useful results: queries, reports, summaries, alerts.
- **Examples:** “Show my last 10 orders”; “Total sales today”; “Patients awaiting reports”.
- **Goal:** Answer questions correctly and quickly to support decisions and automation.

4) Share.

- **What it is:** Delivering processed results to the right people/systems (apps, dashboards, exports).
- **Examples:** Emailing a monthly statement, showing a delivery status in the app, exporting a CSV for analysis.
- **Goal:** Provide timely, role-appropriate access (customer vs. staff) while keeping data consistent.

5) Secure.

- **What it is:** Protecting data during all stages with authentication, authorization, encryption, backups, and audits.
- **Examples:** Only doctors can view certain medical records; passwords are stored hashed; nightly backups are tested.
- **Goal:** Keep data confidential, intact, and available—even during failures or attacks.

Running mini-case: College Library.

1. **Capture:** A student fills a membership form; a librarian scans a book's barcode for checkout.
2. **Store:** The system inserts/updates rows in Student, Book, and Issue tables.
3. **Process:** A query lists all books due this week and calculates fines for overdue items.
4. **Share:** The student sees their current loans in the app; the librarian sees a due-date report.
5. **Secure:** Student data is accessible only to authorized staff; daily backups protect against data loss.

Key takeaways.

- Databases support everyday tasks by keeping data accurate, available, and safe.
- The *capture* → *store* → *process* → *share* → *secure* cycle is a simple mental model you can apply to any information system.
- As systems grow beyond a single user or file, a DBMS becomes essential for performance, concurrency, and reliability.

2.2. Data, Information, and Knowledge

Every digital system works with **data**, turns it into **information**, and, over time, helps us build **knowledge**. In simple words:

- **Data** = raw facts or observations (numbers, words, timestamps) without interpretation.
- **Information** = processed/organized data that answers a specific question.
- **Knowledge** = reusable rules, patterns, or experience that guide decisions.

Term	Meaning (Simple)	Example (Real-Life)
Data	Raw facts or numbers without meaning	25, 30, 28 (temperature readings)
Information	Data that is organized and has meaning	“The average temperature in Delhi today is 28°C”
Knowledge	Understanding from information to make decisions	“It’s 28°C in Delhi, so wearing light woolen clothes is enough”

Everyday intuition

- A list of temperatures recorded every hour (e.g., 31, 33, 36, 35 °C) is **data**.
- “Average temperature today is 34 °C” is **information**.
- “If average summer temperature exceeds 33 °C, start irrigation early” is **knowledge**.

DBMS angle (beginner-friendly)

- **Data in a DBMS:** stored as rows and columns (tuples and attributes) with types (e.g., INT, DATE, VARCHAR).
- **Information from queries:** SQL queries summarize, filter, or join data to answer questions.
- **Knowledge as rules/patterns:** business rules (constraints, triggers), analytics insights, or learned models that guide actions.

Quick comparison

Aspect	Data	Information/ Knowledge
Meaning	Raw facts, unprocessed	Interpreted, answers “who/what/when/how much/what if”
Form	Tables, files, logs	Reports, KPIs, rules, dashboards, alerts
Example	OrderID=1057, Amount=1499, Date=2025-08-22	“Total sales today: 2,45,890 INR”; “Reorder if stock < 10”
DBMS tools	Data types, tables, indexes	SELECT/JOIN/GROUP BY, views, constraints, triggers
User value	Potential	Actionable

Note: “Knowledge” often includes reusable *rules or models*, e.g., a rule like “Block transactions over a limit without OTP” or a model that predicts “churn risk”. In a database course, we mostly operationalize knowledge as *constraints, stored procedures, triggers, and documented policies* built on information.

From data to information: a tiny SQL example

Suppose we have a table:

```
Orders(OrderID, CustomerID, Amount, OrderDate, Status)
```

Questions and queries:

- **Q:** How many orders were placed today?
A (information):

```
SELECT COUNT(*)
FROM Orders
WHERE OrderDate = CURRENT_DATE;
```

- **Q:** What is today’s total sales amount?
A (information):

```
SELECT SUM(Amount) AS TotalSales
FROM Orders
WHERE OrderDate = CURRENT_DATE
AND Status = 'PAID';
```

From information to knowledge: simple rules

- **Restock rule (knowledge):** If `Inventory.StockQty < Inventory.ReorderPoint`, create a purchase order.
- **Data integrity rule (knowledge):** `Order.Amount` must be non-negative; `Order.CustomerID` must exist in `Customer` (foreign key).
- **Actionable threshold (knowledge):** If average delivery time > 3 days this week, alert the logistics manager.

Where does metadata fit?

Metadata is “data about data”—column names, types, sizes, constraints, default values, table relationships, and indexes. In a DBMS, metadata lives in the *catalog* or *data dictionary*, helping the system (and humans) interpret and manage data correctly. Example:

```
ProductID INT PRIMARY KEY,
Name VARCHAR(80) NOT NULL,
Price DECIMAL(10,2) CHECK (Price >= 0)
```

Measurement error and uncertainty (gentle view)

Real-world data is rarely perfect:

- **Entry errors:** Typos, missing values (NULLs), wrong formats.
- **Sensor noise:** Device drift or rounding (e.g., temperature rounded to nearest degree).
- **Timing issues:** Late-arriving records, clock differences.

How DBMS helps (at an introductory level):

- **Validation:** Types, NOT NULL, CHECK constraints reduce bad data.
- **Defaults & ranges:** Provide safe fallbacks; restrict impossible values.
- **Provenance:** Store timestamps and sources to judge reliability later.

Running mini-case: College Library

- **Data:** Each book issue/return event with StudentID, BookID, Date.
- **Information:** “25 books are due this week”; “Top 3 most borrowed titles this semester”.
- **Knowledge:** “If a student has > 3 overdue books, block new issues”; “Increase copies of frequently borrowed books before exams”.

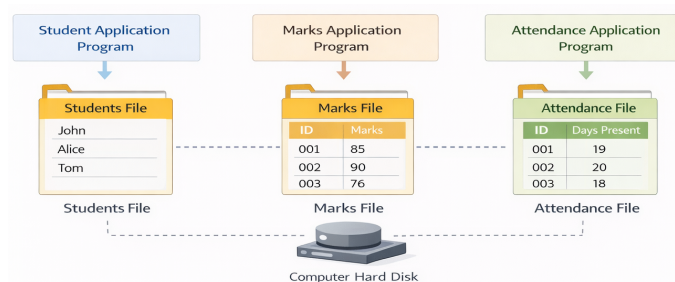
Key takeaways

- Data becomes valuable only after it is *organized* (information) and then *converted into rules or patterns* (knowledge).
- DBMS features (schemas, constraints, queries) help transform data into trustworthy information and support knowledge-based actions.
- Always consider data quality and context; poor input leads to misleading information and weak decisions.

3. File System Approach (Before DBMS)

Before the introduction of DBMS, data was stored and managed using the file system approach. In this method, each application program had its own separate files to store data. These files were usually stored on a computer's hard disk and managed using basic operating system commands.

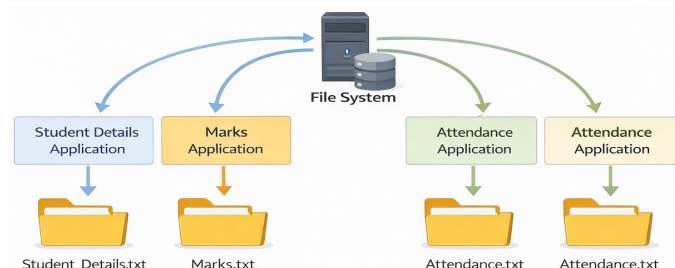
For example, a school might have one file for student details, another file for marks, and another for attendance. Each file worked independently and was handled by separate programs.



3.1. What Are File Systems?

A file system is a basic method of storing data in files on a storage device. Each file contains records, and programs are written to read, write, and update these files. There is no central control system to manage relationships between different files.

In simple words, a file system is just a collection of files stored on a computer.



3.2. Drawbacks of File Systems

The file system approach has many problems.

First, it causes data duplication. The same data is stored in multiple files. This wastes storage space and creates confusion.

Second, it becomes difficult to update data. If a student changes their address, the update must be done in many files. If one file is not updated, incorrect data remains.

Third, data security is weak. File systems do not provide proper permission control. Anyone with file access can view or modify data.

Fourth, file systems do not support multiple users efficiently. If two users try to access the same file at the same time, errors or data loss can occur.

Finally, searching and processing data is slow. There is no powerful query language like SQL, so programmers must write long code to retrieve information.

3.3. Why DBMS Is Better Than File Systems

A DBMS is a modern and advanced way to store and manage data. Unlike file systems, DBMS stores all data in a centralized database, making it easier to access and control.

DBMS reduces data redundancy by storing data only once and sharing it across applications. This saves space and improves accuracy.

It provides strong security features, where different users can have different permissions. Some users can only view data, while others can update or delete it.

DBMS supports multiple users at the same time without conflicts. It uses special techniques to prevent data inconsistency.

DBMS also provides backup and recovery, so data is not lost during system crashes.

Finally, DBMS allows users to retrieve data easily using SQL queries, which makes searching, sorting, and analyzing data very fast.

In Simple Words

File systems are old, slow, and unsafe.

DBMS is modern, fast, secure, and reliable.

File System vs DBMS (Shop Example)

File System (Shop Example)

A shop keeps:

- Customer list in one notebook
- Orders in another notebook
- Payments in another notebook

Problems:

- Same customer name written many times
- Hard to find old records
- Mistakes are common

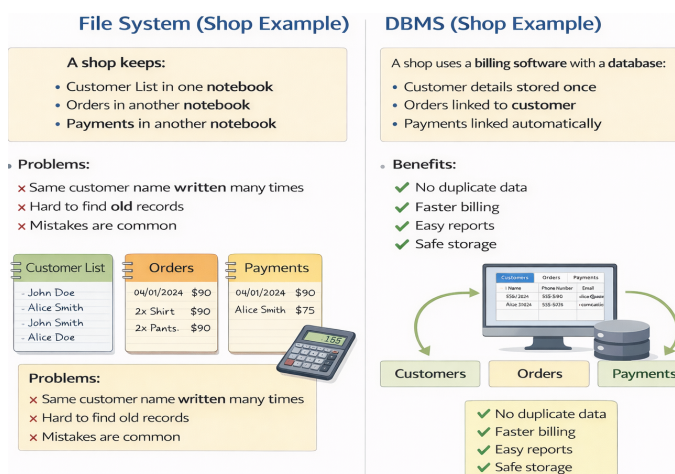
DBMS (Shop Example)

A shop uses a billing software with a database:

- Customer details stored once
- Orders linked to customer
- Payments linked automatically

Benefits:

- No duplicate data
- Faster billing
- Easy reports
- Safe storage



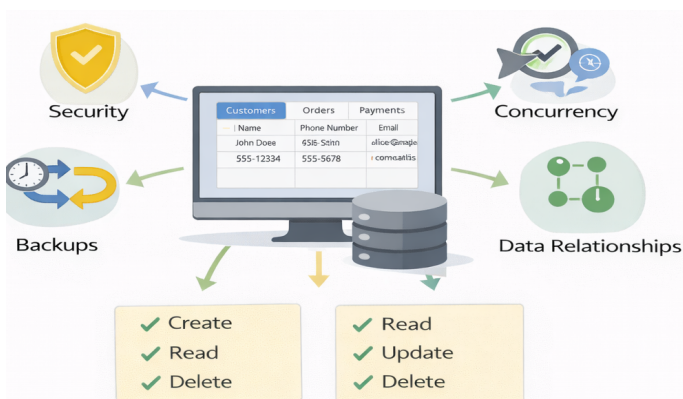
4. DBMS (Database Management System)

What is a DBMS?

A Database Management System (DBMS) is software that stores data in a structured way and allows users to create, read, update, and delete data efficiently. It also manages security, concurrency, backups, and relationships between data.

A DBMS is a software that helps us:

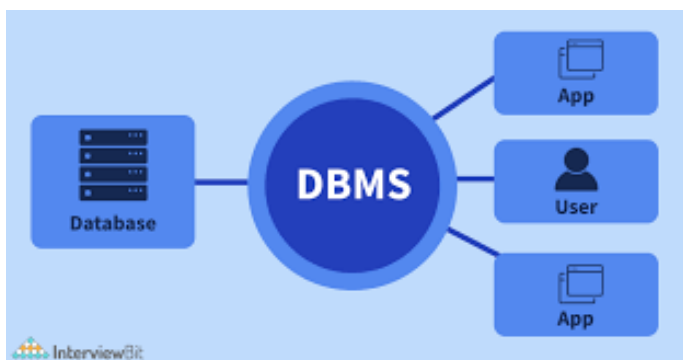
- Store data
- Retrieve data
- Update data
- Delete data
- Manage data



Examples: MySQL, Oracle, SQL Server, PostgreSQL.



It allows users to create, update, and query databases efficiently. It ensures data integrity, consistency, and security across multiple users and applications. It reduces data redundancy and inconsistency through centralized control. It supports concurrent data access, transaction management, and automatic backups.



4.1. Components of DBMS Applications

Any DBMS-based application is made up of six main components that work together to store, manage, and use data properly. These components help the system run smoothly and efficiently.

4.1.1. Hardware

Hardware refers to the physical devices that are used with a DBMS. These include computers, servers, hard disks, keyboards, monitors, printers, and network devices. Hardware is used to store data, process information, and take input from users. For example, the hard disk of a computer stores the database files, and the server processes requests from users.

4.1.2. Software

Software includes the actual DBMS programs such as MySQL, Oracle, and PostgreSQL. It also includes the operating system, network software, and application programs. This software helps users store, retrieve, update, and delete data. It also converts user commands into actions that the system can understand.

4.1.3. Data

Data is the main reason a DBMS exists. It is the collection of facts and information stored in the database. There are two types of data:

- **Operational Data** – Real data like name, age, marks, salary, etc.
- **Metadata** – Data about data, such as file size, data type, and time of storage.

For example, a student's name is operational data, while the type of that name (text) is metadata.

4.1.4. Procedures

Procedures are the rules and instructions that explain how to use the DBMS properly. These include steps for logging in, entering data, validating input, taking backups, controlling access, and generating reports. Procedures help keep the system secure, organized, and consistent.

4.1.5. Database Access Language

This is the language used to communicate with the database. It helps users create, read, update, and delete data. The most common language is SQL (Structured Query Language).

There are two main types:

- **DDL (Data Definition Language)** – Used to create or change tables (CREATE, ALTER, DROP)
- **DML (Data Manipulation Language)** – Used to insert, update, and delete data (INSERT, UPDATE, DELETE)

4.1.6. People

People are the users who interact with the DBMS at different levels.

- **Database Administrators (DBA)** manage security, performance, and user permissions.
- **Developers** create applications using the database.
- **End Users** use the applications to access data, such as students, teachers, or employees.

4.1.7. (Memorable)

Hardware → Physical devices

Software → DBMS programs

Data → Stored information

Procedures → Rules to use the system

Access Language → SQL commands

People → Users of the system

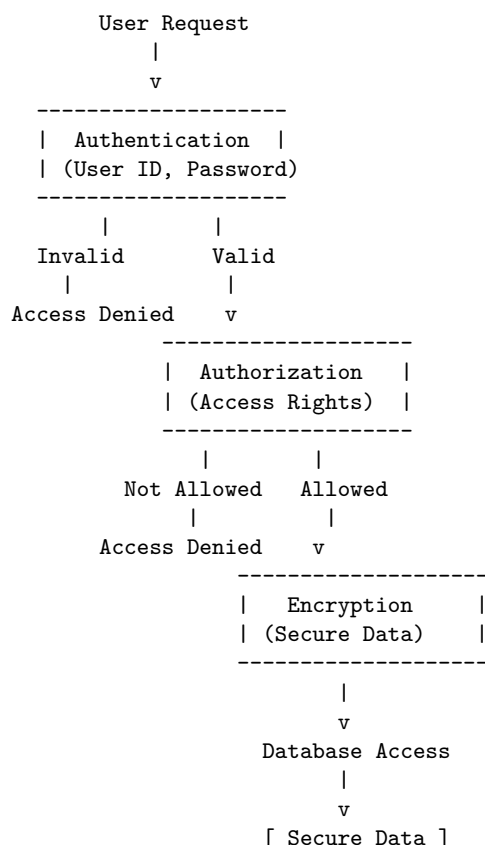
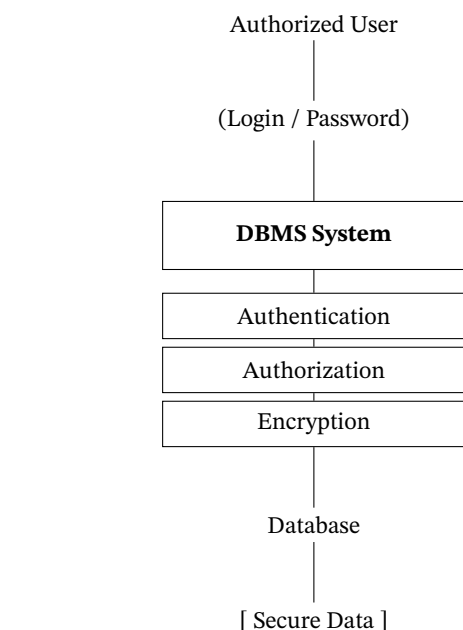
4.2. Advantages of Database Management System (DBMS)

A **Database Management System (DBMS)** is a collection of inter-related data and software programs used to store, manage, access, and manipulate data efficiently. It provides secure, reliable, and organized data handling compared to traditional file systems.

Advantages of DBMS

1. Data Security

DBMS improves data security by using access control, authentication, and encryption. It ensures that only authorized users can access data and reduces the risk of data breaches as the number of users increases.



Working Flow

When a user tries to access data, the request is first checked through **authentication**, where the DBMS verifies the user's login credentials such as username and password. If authentication fails, access is denied immediately.

If authentication is successful, the request moves to **authorization**, where the DBMS checks whether the user has permission

to access the requested data. If the user does not have the required privileges, access is denied again.

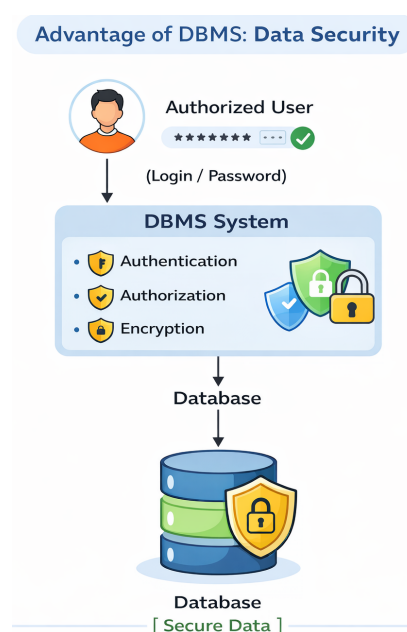
If authorization is granted, the data is accessed through **encryption mechanisms**, ensuring that the data remains secure during storage and transmission. Finally, the authorized user is allowed to access the secure data stored in the database.

Key Security Concepts :-

Authentication : Who you are

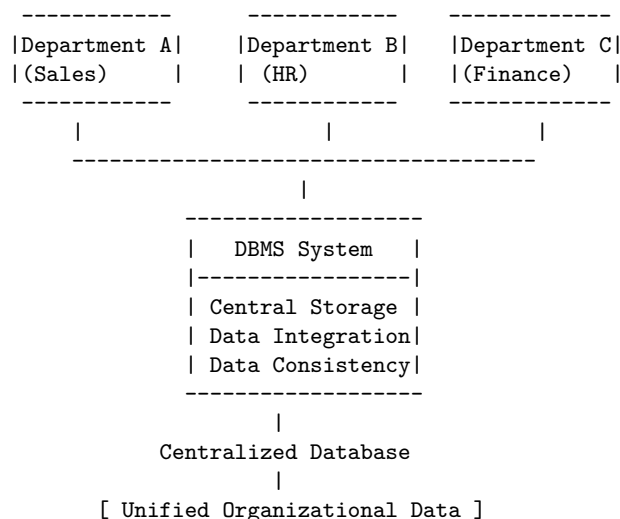
Authorization : What you are allowed to do

Encryption : How data is protected



2. Data Integration

DBMS integrates data from different departments into a centralized database. This provides a unified view of organizational data and helps in understanding how one part of the organization affects another.



Advantage of DBMS: Data Integration



3. Data Abstraction

DBMS hides complex internal operations from users through different levels of abstraction. Users can interact with data easily without knowing how it is stored or processed internally.

User / Application Programs

```

+-----+
| External Level |
| (User Views)  |
+-----+

```

```

+-----+
| Conceptual    |
| Level        |
| (Logical View)|
+-----+

```

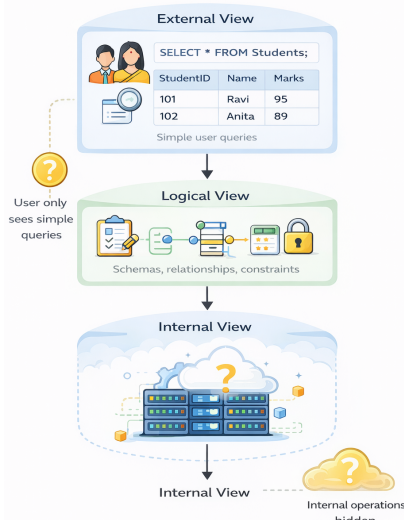
```

+-----+
| Internal Level|
| (Physical    |
| Storage)     |
+-----+

```

Physical Database

Advantage of DBMS: Data Abstraction



4. Reduction in Data Redundancy

DBMS minimizes duplicate data by storing information in a single location. This saves storage space and improves data consistency.

Example: If the same student record appears twice, DBMS removes the duplicate entry.

WITHOUT DBMS (File System)

Student File A			Student File B		
ID	Name		ID	Name	
101	Ravi		101	Ravi	

Duplicate Data Stored

WITH DBMS

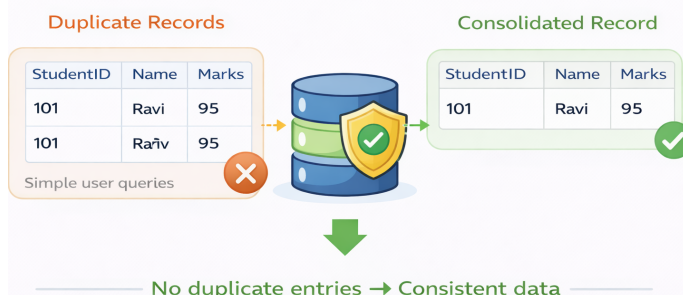
Central Database		
ID	Name	
101	Ravi	

Single copy of data

No redundancy

Improved consistency

Advantage of DBMS: Reduction in Data Redundancy



5. Data Sharing

DBMS allows multiple users and applications to access and share data simultaneously, improving productivity and collaboration.

DATA SHARING IN DBMS

Central Database		
Student-System	Teacher-System	Admin-System
Access-same data	Access-same data	Access-same data

Multiple users → One database

Shared data → Higher productivity

Advantage of DBMS: Data Sharing



6. Data Consistency and Accuracy

DBMS enforces integrity constraints to maintain correct and consistent data. Any update made to data is reflected everywhere, reducing errors.

DATA CONSISTENCY AND ACCURACY

Central Database		
StudentID	Name	
101	Ravi	

Student Portal	Teacher System	Admin System
Shows: Ravi	Shows: Ravi	Shows: Ravi

One update → Reflected everywhere
No inconsistency
Accurate data

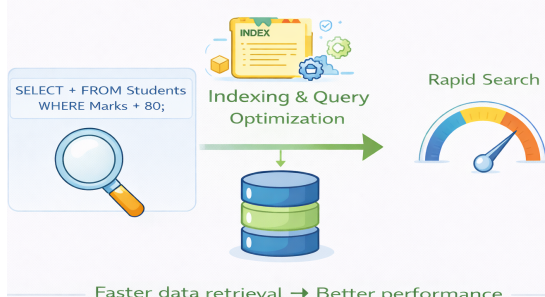
7. Data Organization

DBMS organizes data in a structured format such as tables, rows, and columns, making data easier to store, retrieve, and manage.

8. Efficient Data Access and Retrieval

DBMS uses indexing and query optimization techniques to retrieve data quickly, even from large datasets, thereby improving system performance.

Advantage of DBMS: Efficient Data Access and Retrieval



9. Concurrency and Atomicity

DBMS supports concurrent access by multiple users. Changes made to data are reflected across the database while maintaining consistency.

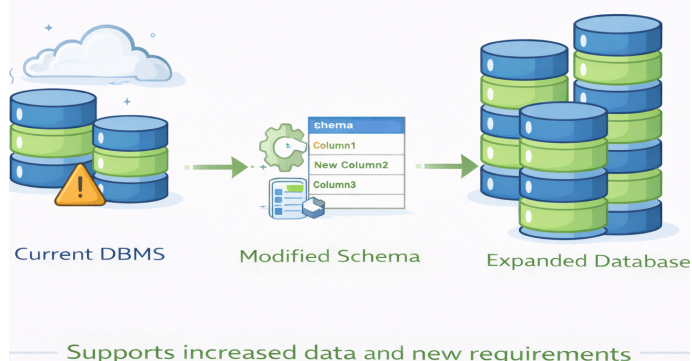
Advantage of DBMS: Data Consistency and Accuracy



10. Scalability and Flexibility

DBMS can handle growing data volumes and changing user requirements. It supports schema modifications and system expansion as needed.

Advantage of DBMS: Scalability and Flexibility



Catchy :

A **Database Management System (DBMS)** offers significant advantages over traditional file-based systems. It ensures data security, integrity, and consistency while providing efficient data access and retrieval. DBMS also reduces data redundancy and supports effective data sharing among multiple users. Due to these benefits, DBMS has become essential for managing data in modern organizations.

4.3. Types of DBMS

There are different types of Database Management Systems (DBMS), and each type is designed for specific kinds of data, different sizes of databases, and various application needs. Some databases are best for structured data, some for large-scale systems, and others for complex relationships. The most commonly used types of DBMS are explained below.

5. Data Models

Data Model is a logical structure of a database. It describes the design of a database to reflect entities, attributes, relationships among data, constraints, etc.

5.1. Introduction to Data Model

One fundamental characteristic of the database approach is that it provides some level of data abstraction. Data abstraction generally refers to the suppression of details of data organization and storage and the highlighting of the essential features for an improved understanding of data.

A **data model** is a collection of concepts that can be used to describe the structure of a database. The Data Model provides the necessary means to achieve this abstraction. By the structure of a database, we mean the **data types, relationships, and constraints** that should hold for the data. Many data models have been proposed, which we can categorize according to the types of concepts they use to describe the database structure.

5.2. Hierarchical Database

A hierarchical database organizes data in a tree-like structure. In this model, each record has only one parent but can have multiple children. This structure is similar to a file system, where folders contain subfolders and files.

Because of its fixed structure, data navigation in hierarchical databases is very fast and predictable. It works well for systems where data naturally follows a hierarchy, such as organizational charts, file systems, or category-based systems.

However, this model is not very flexible. It becomes difficult to modify the structure or handle complex many-to-many relationships. A well-known example of a hierarchical database is IBM Information Management System (IMS).

In this model, each record follows a **parent-child relationship**, where one parent can have multiple children, but each child has only one parent.

The data is stored in different levels, similar to an organizational chart.

Structure of Hierarchical DBMS

- The top level is called the **root**.
- Each parent record can have multiple child records.
- Each child record is linked to only one parent.
- Relationships are represented using pointers or links.

Example:

College → Department → Students

A college can have many departments, and each department can have many students.

Where It Is Best Used Hierarchical databases are suitable for:

- Organizational charts
- File systems
- Category-subcategory data
- Directory structures

Advantages of Hierarchical DBMS

- Simple and easy to understand.
- Fast data access due to predefined relationships.
- Strong data integrity.
- Suitable for one-to-many relationships.

Disadvantages of Hierarchical DBMS

- Complex to implement and maintain.
- Does not support many-to-many relationships.
- Data redundancy may occur.
- Lack of flexibility; structure changes are difficult.

Applications of Hierarchical DBMS

- File systems
- Banking systems
- Telephone directory systems
- IBM Information Management System (IMS)

Conclusion

Hierarchical DBMS is best suited for applications where data follows a strict one-to-many relationship. However, due to its inflexibility and limitations, it is less commonly used in modern database systems.

Real-Life Applications of Hierarchical DBMS

Banking System

In banks, customer data is organized in a hierarchy. A bank has branches, each branch has accounts, and each account has transactions. This parent-child structure makes hierarchical DBMS suitable.

Organization Structure

Companies store employee data hierarchically. An organization has departments, departments have teams, and teams have employees. Each employee reports to one manager.

File System in Computers

Computer file systems follow a hierarchical structure. A root directory contains folders, subfolders, and files. Each file belongs to only one folder.

Telephone Directory System

Telephone directories are arranged hierarchically. A country contains states, states contain cities, and cities contain phone numbers.

Library Management System

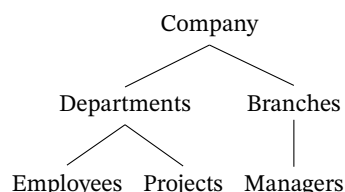
Libraries organize data as library → sections → shelves → books. Each book belongs to a single shelf.

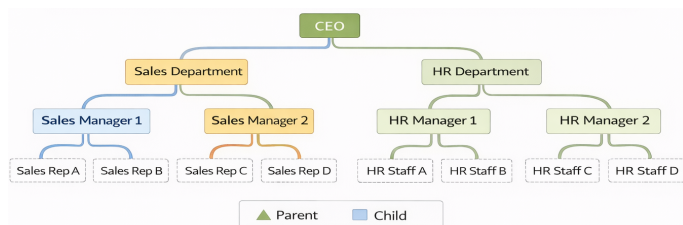
University / College System

A university has colleges, colleges have departments, departments have courses, and courses have students. This strict one-to-many relationship fits the hierarchical DBMS model.

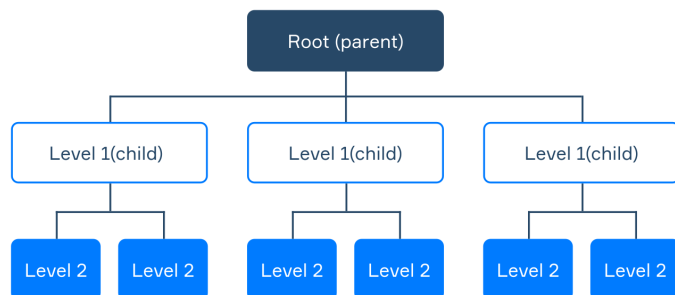
Example of Hierarchical Database IBM Information Management System (IMS)

This system is widely known for using the hierarchical database model, especially in large enterprise and legacy systems. **Structure Example:**





The Hierarchical Database Model



5.3. Network Database

The network database model is an improvement over the hierarchical model. It allows more complex relationships between data. In this system, a single child record can have multiple parent records. This makes it suitable for representing many-to-many relationships.

Data in a network database is represented using records and sets, where sets define how records are related. This structure makes the model more flexible than the hierarchical database and useful for applications that involve complex connections between data.

Network databases are often used in telecommunication systems, airline reservation systems, and complex inventory systems. Examples include Integrated Data Store (IDS) and TurboIMAGE.

Structure of a Network Database

- **Records** → Store actual data (like rows)
- **Sets** → Define relationships between records
- A set links one record type with another
- Overall structure looks like a graph or web, not a tree

Comparison with Hierarchical Model

- **Hierarchical:** One parent → many children
- **Network:** Many parents ↔ many children

Why Network Database Is Useful

- Handles complex relationships
- Less data duplication
- More flexible than hierarchical databases
- Suitable for interconnected data

Applications of Network Database Network databases are commonly used in systems where data has multiple connections, such as:

- Telecommunication systems
- Airline reservation systems
- Complex inventory and supply-chain systems

Examples of Network Databases

- Integrated Data Store (IDS)
- TurboIMAGE

Limitation (for Students to Remember)

- Database design is complex
- Difficult to maintain
- Requires specialized programming knowledge

Real-Life Applications of Network DBMS

Telecommunication Systems

Telecommunication databases store information where one customer can use multiple services, and each service can be used by many customers. This many-to-many relationship makes Network DBMS suitable.

Social Networking Platforms

Users are connected to multiple users, groups, and pages. Each user can have many connections, making the network DBMS an ideal choice.

Airline Reservation System

Passengers can book multiple flights, and each flight can have many passengers. Such complex relationships are efficiently handled by Network DBMS.

University Database System

Students can enroll in multiple courses, and each course can have multiple students. Network DBMS supports this many-to-many relationship effectively.

Supply Chain Management

Suppliers provide multiple products, and products may be supplied by multiple suppliers. This interconnected data structure fits the network model well.

Manufacturing Systems

A product may use multiple components, and a component can be used in multiple products. Network DBMS efficiently handles such complex relationships.

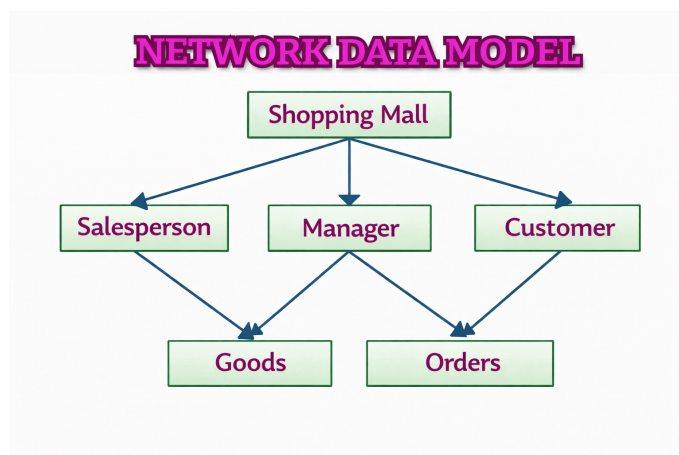
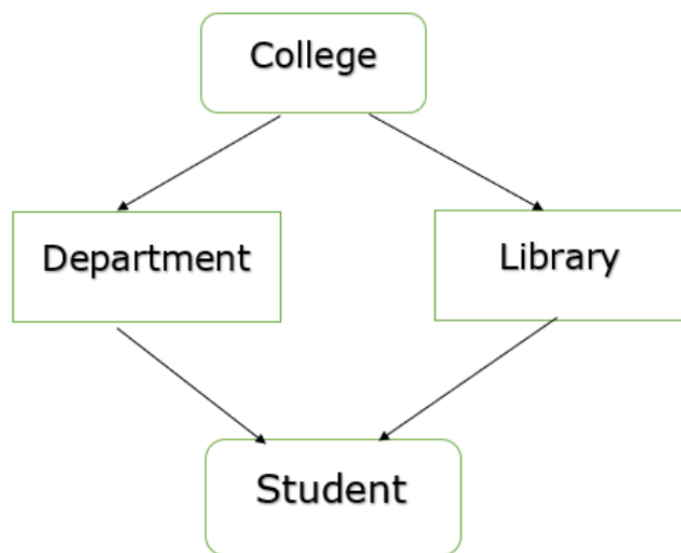


Diagram Representation

Below is an example of a network data model:



Relation DBMS : A **Relational Database Management System (RDBMS)** is a type of DBMS in which data is stored in the form of **tables** (also called relations). Each table consists of **rows** (tuples) and **columns** (attributes). Relationships between different tables are established using **primary keys** and **foreign keys**.

The relational model was proposed by **E. F. Codd** and is based on the mathematical concepts of relations and set theory.

Main Features of RDBMS

- Data is stored in the form of tables.
- Each table has a **primary key** to uniquely identify records.
- **Foreign keys** are used to create relationships between tables.
- Supports **Structured Query Language (SQL)**.
- Ensures data integrity and consistency.
- Follows **ACID properties** (Atomicity, Consistency, Isolation, Durability).

Example

Students Table

StudentID	Name	CourseID
101	Amit	C01
102	Priya	C02

Courses Table

CourseID	CourseName
C01	BCA
C02	BBA

Here, **CourseID** in the Students table acts as a **foreign key** that connects both tables.

Real-Life Applications of RDBMS

- Banking systems – Managing customer accounts and transactions.
- Hospital management systems – Managing patients, doctors, and medical records.
- University systems – Managing students, courses, and examination results.
- E-commerce websites – Managing products, orders, and customers.
- Library management systems – Managing books and members.

Advantages of RDBMS

- Reduces data redundancy.
- Maintains data consistency.
- Easy data retrieval using SQL.

- High security and access control.
- Supports multiple users simultaneously.

Object-Oriented DBMS : An **Object-Oriented Database Management System (OODBMS)** is a type of DBMS that stores data in the form of **objects**, similar to object-oriented programming languages such as Java and C++.

It combines traditional database capabilities with object-oriented concepts like **classes**, **objects**, **inheritance**, and **encapsulation**.

Key Features of OODBMS

- **Objects** – Real-world entities are represented as objects.
- **Classes** – Objects with similar properties and behavior are grouped into classes.
- **Attributes** – Properties or characteristics of an object.
- **Methods** – Functions that define the behavior of an object.
- **Inheritance** – A new class can inherit properties and methods from an existing class.
- **Encapsulation** – Data and methods are combined into a single unit (object).

Example

Class: Student

- **Attributes:** RollNo, Name, Course
- **Methods:** Register(), DisplayDetails()

Object Example:

```

Student1:
RollNo = 101
Name = "Amit"
Course = "BCA"
  
```

Real-Life Applications of OODBMS

- Computer-Aided Design (CAD) systems
- Multimedia systems
- Scientific research databases
- Telecommunication systems
- Artificial Intelligence and Expert systems

Advantages of OODBMS

- Supports complex data types such as images, audio, and video.
- Enables code reusability through inheritance.
- Provides better representation of real-world objects.
- Reduces impedance mismatch between database and programming languages.

Disadvantages of OODBMS

- Complex system design.
- Not as widely used as RDBMS.
- Lack of standardization.
- Higher implementation and maintenance cost.

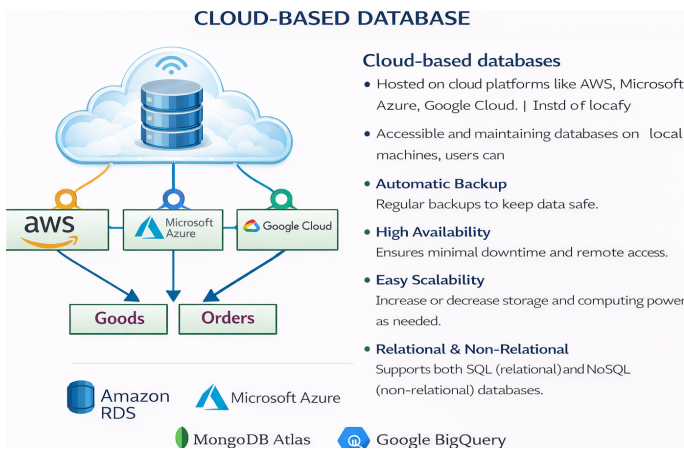
5.4. Cloud-Based Database

Cloud-based databases are hosted on cloud platforms such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud. Instead of installing and maintaining databases on local machines, users can access them over the internet.

These databases provide features like automatic backup, high availability, remote access, and easy scalability. Users can increase or decrease storage and computing power as needed. Cloud databases can be either relational (SQL) or non-relational (NoSQL).

They reduce the need for manual maintenance because the cloud service provider takes care of updates, security, and performance management. Cloud databases are widely used in modern applications that require global access and real-time data processing.

Examples include Amazon RDS for SQL-based databases, MongoDB Atlas for NoSQL databases, and Google BigQuery.



| (Database Server) |

Examples of Cloud Databases

- Amazon Web Services – Amazon RDS
- Google Cloud – Cloud SQL
- Microsoft Azure – Azure SQL Database
- MongoDB Inc. – MongoDB Atlas

Types of Cloud Databases

- **Public Cloud Database** – Managed by a third-party cloud provider.
- **Private Cloud Database** – Used exclusively by a single organization.
- **Hybrid Cloud Database** – Combination of public and private cloud databases.

How It Works

- The user sends a request through an application.
- The cloud server processes the query.
- The result is sent back to the user through the internet.

Advantages

- Accessible from anywhere.
- Highly scalable.
- Automatic backup and recovery.
- Reduced hardware and maintenance cost.
- High availability.

Disadvantages

- Requires a continuous internet connection.
- Security concerns if not properly managed.
- Recurring subscription cost.
- Limited control over infrastructure.

Real-Life Applications

- Online shopping websites
- Banking systems
- Social media platforms
- E-learning platforms

A **Cloud Database** is a database that runs on a cloud computing platform instead of a local server or personal computer. It is hosted and managed by a cloud service provider, and users can access it through the internet.

Meaning

In a cloud database, data is stored on remote cloud servers and can be accessed anytime and anywhere using an internet connection.

Popular Cloud Platforms Popular cloud platforms include:

- Amazon Web Services (AWS)
- Microsoft Azure
- Google Cloud

How It Works

- The database runs on cloud servers
- Users access it remotely via the internet
- No need to install or manage hardware locally

The cloud service provider handles:

- Server management
- Updates and patches
- Security
- Performance tuning

Key Features of Cloud-Based Databases

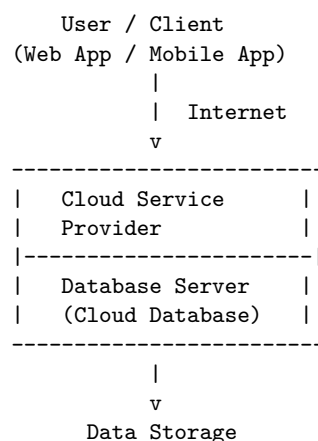
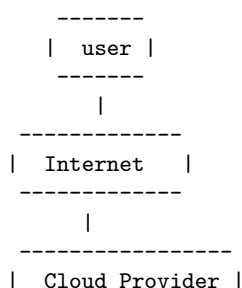
- Automatic backup and recovery
- High availability (less downtime)
- Easy scalability (increase or decrease storage and power anytime)
- Remote and global access
- Real-time data processing

Types of Cloud Databases Cloud databases can be:

- Relational (SQL)
- Non-Relational (NoSQL)

This gives flexibility to choose the right database based on application needs.

Structure :



Client<->Internet<->Cloud Database Server<->Storage

Why Cloud Databases Are Popular

- No manual maintenance
- Cost-effective (pay only for what you use)
- Ideal for modern, large-scale applications
- Supports global users easily

Where Cloud Databases Are Used

- Web and mobile applications
- E-commerce platforms
- Real-time analytics systems
- Applications needing worldwide access

Examples of Cloud-Based Databases

- Amazon RDS – SQL-based cloud database
- MongoDB Atlas – NoSQL cloud database
- Google BigQuery – Large-scale analytics

5.5. NoSQL DBMS

NoSQL databases are designed for handling very large amounts of data and for situations where traditional relational databases become slow or restrictive. Unlike RDBMS, NoSQL databases do not store data in tables with fixed structures. Instead, they use flexible formats such as key-value pairs, documents, graphs, or column-based storage.

This flexibility makes NoSQL databases highly scalable and suitable for unstructured or semi-structured data, such as social media posts, videos, images, and sensor data. They are widely used in modern web applications, big data systems, and real-time analytics.

Some common examples of NoSQL databases are MongoDB, Cassandra, DynamoDB, and Redis.

Data Models in NoSQL NoSQL databases store data in different formats depending on the use case:

Key-Value Store Data stored as key : value

Very fast access

Example: userID → userData

Document Store Data stored as JSON-like documents

Flexible structure (fields can vary)

Column-Based Store Data stored column-wise instead of row-wise

Efficient for large-scale analytics

Graph Database Data stored as nodes and relationships

Ideal for social networks and recommendation systems

Types of Data Handled NoSQL databases are commonly used for:

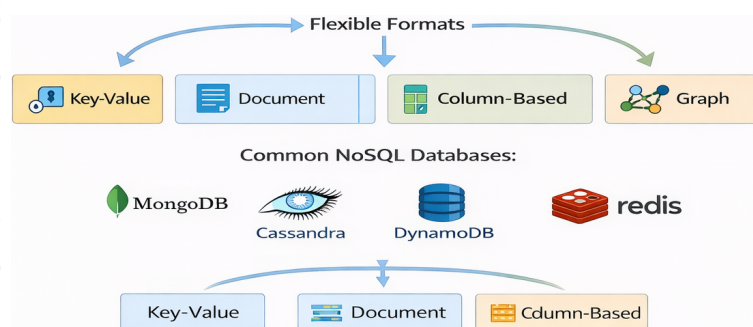
- Social media posts
- Images and videos
- Sensor and IoT data
- Logs and real-time streams

Popular NoSQL Databases

- MongoDB – Document-based
- Cassandra – Column-based
- DynamoDB – Key-value store
- Redis – In-memory key-value store

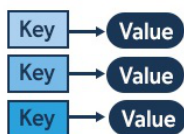
Where NoSQL is Used

- Modern web applications
- Big data platforms
- Cloud-based systems
- Real-time analytics and streaming applications

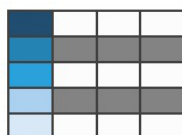


NoSQL

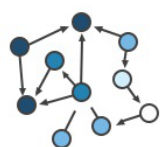
Key-Value



Column-Family



Graph



Document



Why NoSQL is Powerful Because of its flexible design, NoSQL offers:

- High scalability (horizontal scaling)
- Handles unstructured and semi-structured data
- Works well with big data
- Supports real-time processing

5.6. Advantages and Disadvantages of DBMS

Advantages of DBMS A Database Management System (DBMS) helps in organizing and managing data in a structured and systematic manner. Data is stored in tables consisting of rows and columns, which makes searching, sorting, and updating data easy and efficient.

Example: In a school database, we can quickly find students of a particular class or update a student's marks without manually checking multiple files.

1. Efficient Data Organization and Management

DBMS stores data in a structured format, making it easy to access and modify.

Example: In a school database:

- Find students of Class 10 instantly.
- Update marks without searching multiple files.
- Sort data alphabetically or numerically.

⇒ This saves time and reduces manual effort.

2. Reduced Data Redundancy

Data redundancy means storing the same data multiple times.

In traditional file systems:

- Student details appear in attendance files.
- Same details appear in marks files.
- Same details appear in fee records.

Leads to:

- × Wastage of storage
- × Data inconsistency

In DBMS:

- ✓ Data is stored once
- ✓ Used everywhere through references
- ✓ Saves storage space
- ✓ Maintains consistency

3. Strong Data Security

DBMS provides access control and authorization mechanisms.

Example:

- Receptionist: Can only view records
- Doctor: Can update medical data
- Manager: Can see financial reports

⇒ Unauthorized users are blocked from sensitive data, protecting confidential information.

4. Data Integrity and Accuracy

Ensures data is correct, valid, and consistent. DBMS uses constraints such as:

- Age cannot be negative
- Price cannot be zero
- Email ID must be unique

Invalid data is rejected, keeping data reliable and trustworthy.

5. Multi-User Support

Allows multiple users to access the same database simultaneously.

Example: In a hospital system:

- Doctors view patient records
- Nurses update vitals
- Staff manages billing

DBMS uses locking and transactions to prevent conflicts and maintain consistency.

6. Backup and Recovery

Provides automatic backup and recovery features. In case of:

- Power failure
 - Software error
 - Hardware fault
- ✓ Data can be restored
✓ System returns to the previous stable state

Important for banks, airlines, and large organizations.

7. Powerful Querying Using SQL

DBMS uses SQL (Structured Query Language) to retrieve specific data.

Examples:

- Students scoring above 90
- Total sales of the month
- Employees hired this year

Queries can be executed quickly and accurately.

8. Data Sharing

Allows sharing between multiple applications such as:

- Mobile App
- Website
- Desktop Software

⇒ Ensures no duplication and consistent data.

Disadvantages of DBMS Although DBMS is powerful, it has some limitations:

1. High Cost

Requires expensive software, powerful servers, and trained professionals, making it costly for small businesses.

2. Complexity

Not easy to learn; users must understand tables, relationships, queries, and transactions, requiring time and training.

3. High Resource Consumption

Consumes more memory, CPU, and storage. For very small applications, DBMS may be unnecessary.

4. Security Risk if Misconfigured

If not configured properly:

Hackers can gain access
Data can be leaked

Security depends on correct setup and maintenance.

5. Performance Overhead

Performs background tasks like security checks, backup, and concurrency control. Simple operations may be slower than a normal file system.

Prepared by Eimple Labs | Distributed under Creative Commons License. This material is for personal use only. Redistribution, replication, or sharing in any form, whether digital or physical, without prior authorization from Eimple Labs, is strictly prohibited.

6. DBMS Architecture

Database system architecture refers to how a database system is designed and how all its components work together to store, manage, and retrieve data. It explains who interacts with the database, how requests move within the system, and how data is stored and processed. It can be thought of as the *blueprint* of a database system.

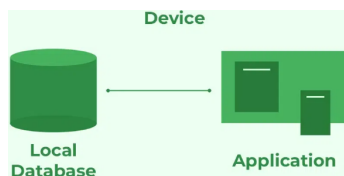
A database system mainly consists of **users**, **DBMS software**, and the **database** where the actual data is stored. Users interact with the system to enter, update, or retrieve data. The DBMS software acts as a bridge between the users and the database by managing requests and controlling how data is accessed and modified. The database is the place where all information is stored in an organized and structured form.

6.1. Single-Tier Architecture

6.1.1. Single-Tier Architecture (1-Tier)

Single-tier architecture means that the **user**, **application**, and **database** are all present on the same system. In this setup, everything runs on one computer, so the user interacts directly with the DBMS, and the DBMS manages the database stored on that same machine. The structure can be understood as:

User → DBMS → Database



with all components located on a single system.

A common example of this architecture is **MS Access** being used on one personal computer for small projects. This type of system is simple to set up and works fast for small applications because there is no network communication involved.

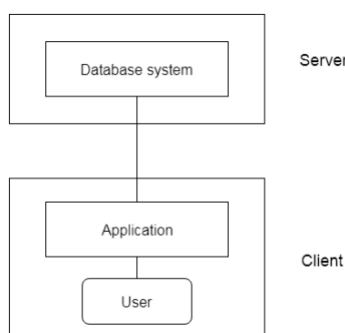
However, it has several disadvantages. It is not very secure, it cannot handle large applications, and it does not support remote access. Because of these limitations, single-tier architecture is mostly used for learning purposes or small standalone systems.

6.2. Two-Tier Architecture

6.2.1. Two-Tier Architecture (2-Tier)

Two-tier architecture is also known as **client-server architecture**. In this design, the application runs on the client machine, while the database is stored on a separate server. The structure can be understood as:

Client (Application) → Database Server



A common example of this architecture is desktop software connected to a **MySQL** server, such as a college library management system. In this system, the client sends an SQL query to the server.

The database server processes the request and sends the result back to the client.

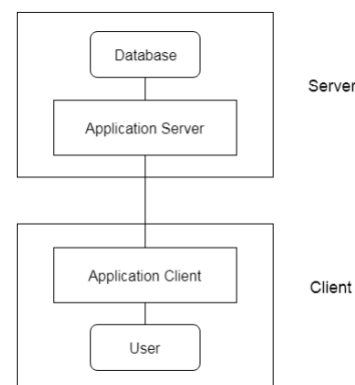
This architecture is better than single-tier systems because data is stored in one central place and multiple users can access it at the same time. However, it has some disadvantages. Since the client communicates directly with the database, security is weaker. The server may also face heavy load when many clients are connected, and managing very large systems becomes difficult.

6.3. Three-Tier Architecture

6.3.1. Three-Tier Architecture (3-Tier)

Three-tier architecture is the most modern and widely used database architecture. In this design, there are three layers between the user and the database. The structure works as:

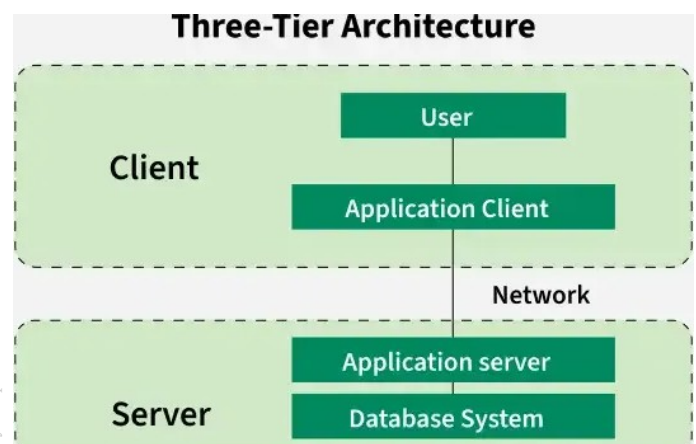
User (Client) → Application Server (Business Logic) → Database Server



The first layer is the **Presentation Layer**, which is the client side. This is what the user sees and interacts with, such as a website, mobile application, or software interface. The second layer is the **Application Layer**, also called the **Business Logic Layer**. This layer processes user requests, applies business rules, and communicates with the database. The third layer is the **Database Layer**, where data is stored and SQL queries are executed.

A good example of this architecture is an **online shopping website**. The user interacts with the website interface. The website sends the request to the application server, which processes the request and then communicates with the database server to store or retrieve data.

This architecture has many advantages. It is very secure because the user does not directly access the database. It is easier to maintain, highly scalable to handle many users, and provides good performance. However, it is more complex to design and costs more compared to simpler architectures.



7. Data Independence

7.1. Data Independence

Data independence means the ability to change the structure of the database without affecting the application programs that use the data. It allows the database design to be modified without rewriting the entire software. This makes the system flexible and easier to maintain. Data independence is mainly of two types: **logical data independence** and **physical data independence**.

Real-Life Example of Data Independence

Consider a **College Management System**.

It stores:

- Students
- Teachers
- Courses
- Marks

Many applications use this database:

- Student portal
- Teacher login
- Admin software

Now we can observe how changes in the database structure can happen without breaking or affecting the existing software applications.

7.2. Logical Data Independence

Logical data independence is the ability to change the **logical structure** of the database without changing the application programs. Logical structure refers to the design of tables, such as adding or removing columns, changing relationships, or modifying constraints. For example, if a new column like **Email** is added to a **Student** table, existing programs should continue working without any changes. Achieving logical data independence is difficult because changes in table structure can affect how data is used by applications.

Real-Life Example of Logical Data Independence

Example: Hospital Management System

A hospital uses software to manage:

- Patient records
- Doctor details
- Appointments
- Bills

Many applications use the same database:

- Reception system
- Doctor dashboard
- Billing system

Step 1: Original Database Design

Patient Table

Patient_ID	Name	Age	Disease
------------	------	-----	---------

Programs are built to:

- Show patient name
- Check age
- View disease details

Everything works fine.

Step 2: Hospital Needs More Information

The hospital decides to store extra details:

- Email
- Blood Group

So the table becomes:

Patient_ID	Name	Age	Disease	Email	Blood_Group
------------	------	-----	---------	-------	-------------

What Happens Now?

- The database logical structure has changed (new columns added).
- Existing applications such as the reception system, doctor system, and billing system still work normally.
- They only use **Patient_ID**, **Name**, **Age**, and **Disease**.
- They do not depend on the newly added columns.

Therefore, there is no need to rewrite the entire hospital software, demonstrating **logical data independence**.

7.3. Physical Data Independence

Physical data independence is the ability to change the **physical storage** of data without affecting the logical structure of the database or the application programs. Physical storage refers to how data is stored on disk, such as file organization, indexing methods, or storage devices. For example, if the database administrator changes the indexing method to improve performance, users and applications should not notice any difference. Physical data independence is easier to achieve compared to logical data independence.

Real-Life Example of Physical Data Independence

Example: Online Shopping Website (e.g., Amazon / Flipkart)

The system stores:

- Customers
- Products
- Orders
- Payments

Millions of users access the system through:

- Mobile app
- Website
- Seller dashboard

All these applications use the same database.

Step 1: Initial Storage

Initially, the database is stored:

- On a normal hard disk
- Without special indexing

The system works, but searching for products is slow.

Step 2: Database Administrator Improves Storage

To improve performance, the Database Administrator (DBA):

- Adds indexes on **Product_ID**
- Changes file organization
- Moves the database from HDD to SSD
- Shifts storage to a cloud server

These are **physical changes** related only to internal storage.

What Happens to Users?

- The website still looks the same
- The mobile app still works the same
- No change in tables
- No change in application programs

The only visible difference is improved performance—the system becomes faster. Users are not aware that any internal storage changes have taken place, demonstrating **physical data independence**.

8. Database Languages

8.1. Data Definition Language (DDL)

DDL stands for **Data Definition Language**. It is a set of SQL commands used to create and manage the structure of a database. DDL commands help in creating, modifying, and deleting database objects such as tables. These commands include CREATE, ALTER, DROP, TRUNCATE, and RENAME. DDL mainly deals with the structure and storage of data, not the actual data stored inside the tables.

Types of DDL Commands

- **CREATE** — Used to create database objects such as databases and tables.
- **ALTER** — Used to change the structure of existing database objects.
- **DROP** — Used to remove database objects permanently.
- **TRUNCATE** — Used to delete all records from a table quickly.
- **RENAME** — Used to change the name of a database object.

Applications of Data Definition Language (DDL)

Data Definition Language (DDL) is used to define, create, and manage the structure of a database. Its main applications include:

- Creating database objects such as tables, views, and indexes.
- Modifying existing database objects by adding or removing columns or changing data types.
- Managing constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, and NOT NULL.
- Managing database permissions and access control.
- Creating indexes to improve query performance.
- Partitioning tables to efficiently handle large volumes of data.

8.2. Data Manipulation Language (DML)

8.3. Data Definition Language (DDL) Commands

8.3.1. 1. CREATE

The **CREATE** command is used to create a table in a relational database. While creating a table, we specify the column names and their data types.

Syntax:

```
CREATE TABLE table_name
(
    column_name1 datatype1,
    column_name2 datatype2,
    column_name3 datatype3
);
```

Here, each column name specifies the name of the column, and the datatype defines what type of data that column can store.

Example:

```
CREATE TABLE Student
(
    Student_id INT,
    Name VARCHAR(100),
    Marks INT
);
```

In this table, Student_id is an integer, Name is a string (text), and Marks is an integer.

8.3.2. 2. ALTER

The **ALTER** command is used to change the structure of an existing table. It can be used to add a column, rename a column, delete a column, or change the datatype or size of a column.

To add a column:

```
ALTER TABLE table_name ADD (column_name datatype);
```

Example:

```
ALTER TABLE Student
ADD (Address VARCHAR(200));
```

This adds a new column named Address to the Student table.

To rename a column:

```
ALTER TABLE table_name
RENAME old_column_name TO new_column_name;
```

Example:

```
ALTER TABLE Student
RENAME Marks TO Age;
```

This changes the column name from Marks to Age.

To drop (delete) a column:

```
ALTER TABLE table_name
DROP (column_name);
```

Example:

```
ALTER TABLE Student
DROP (Age);
```

This removes the column Age from the table.

To modify a column:

```
ALTER TABLE table_name
MODIFY (column_name datatype);
```

Example:

```
ALTER TABLE Student
MODIFY (Name VARCHAR(300));
```

This changes the size of the Name column.

8.3.3. 3. TRUNCATE

The **TRUNCATE** command removes all records from a table but does not delete the table structure.

Syntax:

```
TRUNCATE TABLE table_name;
```

Example:

```
TRUNCATE TABLE Student;
```

This deletes all rows from the Student table, but the table itself still exists.

8.3.4. 4. DROP

The **DROP** command completely removes a table from the database. It deletes both the data and the table structure.

Syntax:

```
DROP TABLE table_name;
```

Example:

```
DROP TABLE Student;
```

This deletes the Student table permanently.

Difference between TRUNCATE and DROP: TRUNCATE removes only the data, while DROP removes both the data and the table itself.

Advantages of DDL

- Helps define and organize the structure of the database.
- Commands like CREATE and ALTER allow changes in tables and other objects.
- Supports constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, and NOT NULL to maintain data accuracy.
- Can improve performance by creating indexes and partitions.
- Mostly standard across SQL databases, making it easy to learn and use.

Disadvantages of DDL

- Many commands like DROP and TRUNCATE cannot be undone and may cause permanent data loss.
- Incorrect usage can delete important data.
- Structural changes in large databases can be complex and may require downtime.
- DDL operations can lock tables during execution, slowing other operations.
- Syntax differences may exist between database systems such as MySQL, Oracle, and PostgreSQL.

8.4. Applications of Data Definition Language (DDL)

Data Definition Language (DDL) is used to define, create, and manage the structure of a database. Its main applications include:

- Creating database objects such as tables, views, and indexes.
- Modifying existing database objects by adding or removing columns or changing data types.
- Managing constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, and NOT NULL.
- Managing database permissions and access control.
- Creating indexes to improve query performance.
- Partitioning tables to efficiently handle large volumes of data.

8.5. Data Control Language (DCL)

DCL stands for **Data Control Language** in SQL. DCL commands are used to control access and permissions in a database. These permissions determine who can access the data and what actions they are allowed to perform, such as creating tables, views, or other database objects. DCL mainly deals with user rights, permissions, and database security.

There are two types of privileges in a database:

- **System Privileges** — Allow users to perform system-level actions such as creating tables or users.
- **Object Privileges** — Allow users to perform actions on specific database objects such as tables or views.

DCL commands are essential for preventing unauthorized access and maintaining database security. They ensure that only authorized users can access or modify data and give the database administrator fine-grained control over permissions.

8.5.1. DCL Commands

The two main DCL commands are GRANT and REVOKE.

8.5.2. GRANT Command

The **GRANT** command is used to give permission to a user to perform specific operations on a database object. For example, an administrator may allow one user to only view data while another user can update data.

Syntax:

```
GRANT privilege_list
ON object_name
TO user_name;
```

8.5.3. REVOKE Command

The **REVOKE** command is used to remove permissions from a user. If a user no longer requires access, the administrator can take back the granted privileges.

Syntax:

```
REVOKE privilege_list
ON object_name
FROM user_name;
```

Common Privileges

Some commonly used privileges include:

- SELECT
- UPDATE
- DELETE
- ALTER
- EXECUTE
- ALL

Advantages of DCL

- Restricts unauthorized access to database data.
- Ensures database security in multi-user environments.
- Provides full control to the database administrator over user permissions.
- Prevents unauthorized users from making unwanted changes.

Disadvantages of DCL

- Controls access only and does not handle data manipulation.
- Incorrect use of GRANT or REVOKE may lead to security risks.
- Managing permissions becomes complex in large databases.
- Cannot perform INSERT, UPDATE, or DELETE operations directly.
- Mainly useful after DDL and DML operations are completed.

8.6. Transaction Control Language (TCL)

TCL stands for **Transaction Control Language**. It is a part of SQL used to manage transactions in a database. A **transaction** is a group of operations that are performed together. TCL helps control whether the changes made in a transaction should be saved permanently or cancelled. It plays an important role in maintaining data integrity and keeping the database consistent.

TCL mainly includes the commands COMMIT, ROLLBACK, and SAVEPOINT.

8.6.1. COMMIT Command

The **COMMIT** command is used to permanently save all changes made during the current transaction. Once a transaction is committed, the changes cannot be undone. It is usually used after a set of INSERT, UPDATE, or DELETE operations to store the final result in the database.

Syntax:

```
COMMIT;
```

8.6.2. ROLLBACK Command

The **ROLLBACK** command is used to undo all changes made during the current transaction that have not yet been committed. It restores the database to its previous state. This command is useful when an error occurs or when changes should not be saved.

Syntax:

```
ROLLBACK;
```

8.6.3. SAVEPOINT Command

The **SAVEPOINT** command is used to create a temporary point within a transaction. Later, the transaction can be rolled back only up to that savepoint instead of cancelling the entire transaction. This provides better control over transaction handling.

Syntax:

```
SAVEPOINT savepoint_name;
```

8.6.4. Working of TCL Commands

TCL commands are used after data manipulation commands such as INSERT, UPDATE, and DELETE. The behavior of TCL commands can be summarized as follows:

- COMMIT saves all changes permanently.
- ROLLBACK cancels all uncommitted changes.
- SAVEPOINT allows partial rollback within a transaction.

Another related command is `SET TRANSACTION`, which is used to control transaction properties such as isolation level.

A transaction usually starts automatically when a DML command is executed, or it can be started manually using `BEGIN TRANSACTION`. All operations remain temporary until `COMMIT` is executed. During a transaction, multiple savepoints can be created. At the end, either `COMMIT` is used to save all changes or `ROLLBACK` is used to undo them. When a transaction ends, all locks and resources are released.

TCL helps maintain the **ACID properties**—**Atomicity, Consistency, Isolation**, and **Durability**—which ensure reliable database operations.

Example

If a student's name is updated from Jolly to Sherlock and `COMMIT` is executed, the change becomes permanent. If `ROLLBACK` is executed instead, the name reverts to Jolly.

If savepoints A, B, and C are created, rolling back to B removes changes after B, while rolling back to A keeps only the earliest changes.

8.7. Real-Life Example of Data Independence

Consider a **College Management System**. It stores:

- Students
- Teachers
- Courses
- Marks

Many applications use this database:

- Student portal
- Teacher login
- Admin software

8.7.1. Logical Data Independence

Logical data independence means changing the logical structure (tables or columns) without affecting application programs.

Initial Student Table:

RollNo	Name	Course
--------	------	--------

Later, new columns are added:

RollNo	Name	Course	Email	Phone
--------	------	--------	-------	-------

Even though the table structure changes, old applications that use only RollNo, Name, and Course continue to work without modification. This demonstrates **Logical Data Independence**.

8.7.2. Physical Data Independence

Physical data independence means changing how data is stored internally without affecting application programs.

Initially, the database may be stored on a hard disk. Later, it may be moved to:

- SSD
- Cloud storage
- Faster storage systems
- Indexed file structures

These changes improve performance, but:

- Student portal still works
- Teacher login still works
- No application code changes are required

This demonstrates **Physical Data Independence**.

9. Database Users

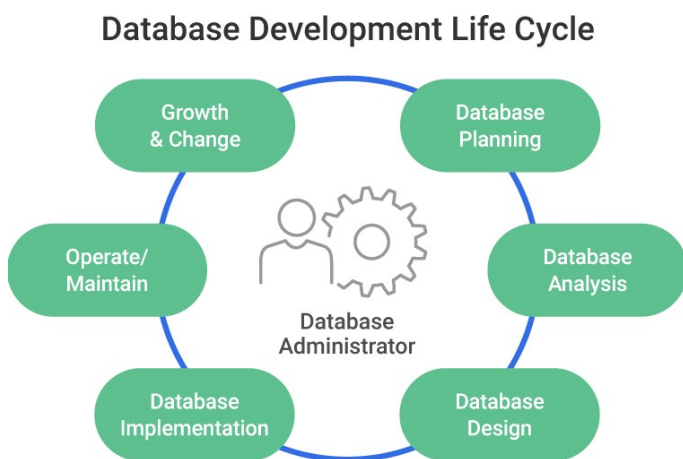
9.1. Database Administrator (DBA)

DBA stands for Database Administrator. A Database Administrator is the person responsible for controlling, maintaining, and managing the database management system. Their main role is to ensure that the database works smoothly, remains secure, and is always available when needed.

A DBA controls who can access the database and authorizes users to protect data from unauthorized access. They handle tasks such as database installation, configuration, capacity planning, and performance monitoring. DBAs also manage both software and hardware resources required for the database system.

The responsibilities of a DBA include database design, migration, security management, troubleshooting issues, taking regular backups, and performing data recovery in case of failures. They also handle problems caused by hardware or software errors and ensure minimal data loss.

Database administration is a critical function in any organization that depends on databases. Overall, the DBA acts as the main controller and caretaker of the entire database system.



9.2. Different Types of Database Users

A **database user** is any person who interacts with a database to store, retrieve, update, or manage data. Users do not usually access raw data files directly; instead, they interact with the database through a **Database Management System (DBMS)**, application programs, or user interfaces.

Based on how users interact with the database system, database users are classified into different categories.

9.2.1. Database Administrator (DBA)

A **Database Administrator (DBA)** is the person responsible for managing and controlling the entire database system. The DBA plays a crucial role in ensuring that the database remains **available, secure, reliable, and efficient** at all times. One of the primary duties of a DBA is to define the database structure (schema) and manage all levels of database architecture.

The DBA is also responsible for:

- Creating and maintaining user accounts.
- Assigning appropriate roles and privileges.
- Controlling access to data so that only authorized users can read or modify it.

In addition, the DBA ensures **data security and integrity** by enforcing rules, constraints, and authorization policies within the database system. Regular database backups are taken by the DBA, and proper recovery procedures are performed in case of hardware failures, software crashes, or data corruption.

The DBA continuously monitors the database to tune performance, ensuring efficient storage and fast data retrieval. When system failures occur, the DBA handles hardware and software issues and repairs any damage caused to the database. The DBA also provides technical support and guidance related to database operations and maintenance.

Because of these responsibilities, the DBA is granted special administrative privileges and can use **Data Control Language (DCL)** commands such as **GRANT** and **REVOKE** to manage user permissions and access rights.

Why the DBA Is Important

Without a DBA, a database system may suffer from:

- Unauthorized access
- Data loss
- Poor performance
- System failures without proper recovery options

Thus, the **Database Administrator** is essential for the smooth, secure, and efficient operation of any database system.

9.3. Developers

Database Developers

Developers are professionals who build software applications that interact with a database. They design and write program code that allows users to easily enter, update, and retrieve data. Developers commonly use programming languages such as Java, Python, C#, or PHP along with SQL to connect applications to the database.

They are responsible for creating forms, reports, and user interfaces, as well as writing backend logic that processes user requests. Developers ensure that applications communicate correctly with the DBMS, handle errors properly, and maintain data accuracy. They often implement business rules, validations, and security checks within the application to ensure reliable and secure database operations.

9.4. End Users

End users are the people who use a database system to perform their daily work. They do not manage the database or write complex programs. Instead, they use applications, software, or user interfaces that are connected to the database to enter, update, or view data.

End users interact with the system in a simple manner, such as filling forms, clicking buttons, or searching for information. For example, a bank clerk updating customer details, a student checking exam results, or a customer booking a train ticket are all end users. They focus only on completing their tasks and do not require technical knowledge of how the database works internally.

10. Database Design Phases

The design of a database application that meets the needs of the enterprise being modeled involves various phases as outlined below:

10.1. Requirement Collection & Analysis

The initial phase of database design is to understand and document the requirements of the database users. The designer fully characterizes the data needs of the prospective database users. The database designer needs to interact extensively with domain experts and users to carry out this task.

The outcome of this phase is the **User Requirement Specification**. In parallel with specifying the data requirements, it is useful to specify the known functional requirements of the application. These consist of the user-defined operations (or transactions) that will be applied to the database, including both retrievals and updates.

10.2. Conceptual Design Phase

Once all the requirements have been collected and analyzed, the next step is to create a **conceptual schema** for the database, using a high-level conceptual data model. This step is called **conceptual design**.

The conceptual schema is a concise description of the data requirements of the users and includes detailed descriptions of the **entity types, relationships, and constraints**; these are expressed using the concepts provided by the high-level data model.

The conceptual schema describes the overall logical structure of the enterprise using conceptual tools. Because these concepts do not include implementation details, they are usually easier to understand and can be used to communicate with non-technical users. This approach enables database designers to concentrate on specifying the properties of the data without being concerned with storage details.

The **Entity-Relationship Model (E-R Model)** is mainly used to represent the conceptual design. The conceptual-design phase results in the creation of an **Entity-Relationship Diagram** that provides a graphical representation of the schema.

10.3. Logical Design Phase

The process of moving from an abstract data model to the implementation of the database using a commercial DBMS proceeds in two final design phases.

In the **logical design phase**, the designer maps the high-level conceptual schema onto the implementation data model of a specific DBMS. The implementation data model is typically the **relational data model**, and this step generally consists of mapping the conceptual schema defined using the entity-relationship model into a relation schema.

10.4. Physical Design Phase

The last step is the **physical design phase**, during which the internal storage structures, indexes, access paths, and file organizations for the database files are specified. The designer uses the resulting system-specific database schema in the subsequent physical-design phase, where the physical features of the database are specified.

1. E-R Model

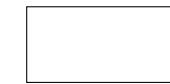
Entity-Relationship Model is a systematic way of describing and defining a business process. An ER model is typically implemented as a database. The main components of the E-R model are:

- Entity Set
- Relationship Set

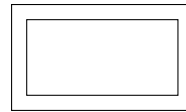
Here are the geometric shapes and their meaning in an E-R diagram:

- **Entity**

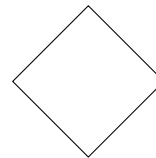
- **Weak Entity Set**



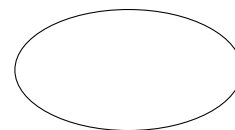
- **Relationship Set**



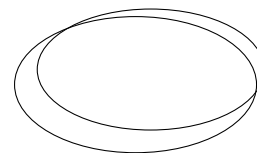
- **Attributes**



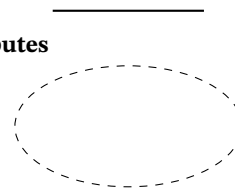
- **Multivalued Attributes**



- **Link attribute to entity set and entity set to relationship set.**



- **Derived Attributes**



- **Total Participation of an Entity in a Relationship Set**



11. Introduction to E-R Model

The Entity-Relationship (ER) Model is a conceptual model used in the early stage of database design. It helps in planning how a database should be structured before it is actually created in a DBMS.

In the ER model, data is represented using entities, attributes, and relationships. An entity is a real-world object or concept, such as a student, teacher, or product. Attributes describe the properties of an entity, like name, age, or ID. Relationships show how different entities are connected to each other.

The main purpose of the ER model is to visually represent the logical structure of a database. It helps database designers clearly understand how data is related and organized. This makes the process of converting the design into actual tables much easier and more accurate.

The E-R model provides three basic concepts to describe the database:

- (a) **Entities**
- (b) **Attributes**
- (c) **Relationship**

11.1. Entity Type, Entity Set and Attributes

11.1.1. Entity

is an object in the real world with an independent existence. It is an object that is *distinguishable* from all the other objects. An entity may be an object with a physical existence / concrete (for example, a particular person, car, house, or employee), or it may be an object with a conceptual existence / abstract (for example, a company, a job, or a university course).

11.1.2. Attributes

are the properties that characterize / describe an entity. Attributes of the *customer* entity are *customer id*, *customer name*, *customer street*, and *customer city*. A particular entity will have a value for each of its attributes.

Customer-Id	312-12-3123
Customer-Name	Jones
Customer-Street	Ranihat
Customer-City	Cuttack

11.1.3. Entity Set:

An **entity set** is a set of entities that share the same properties or attributes. The set of all persons who are **customers** at a given bank can be defined as the entity set *customer*. Each entity in the entity set has its own value for each attribute.

An entity is a single object of an entity type. The collection of all entities of the same type is called an **entity set**.

Example:

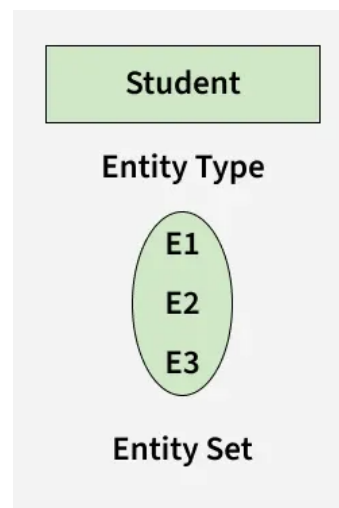
- “Student” is an entity type
- A particular student like E1 is an entity (instance)
- The group of all students is the entity set

Entity Type: An **Entity Type** defines the collection (set) of entities that have the same attributes. It defines the type of the entity, i.e., the attributes that an entity possesses and the relationship in which the entity participates. Each entity type in the database is described by its name and attributes. The set of entities of a particular entity type in the database at any point in time is called an **entity set**. An entity type describes the **schema** or **intension** for a set of entities that share a structure. The entity set is also called the **extension** of the entity type.

Types of Entities

There are two main types of entities in the ER Model:

- Strong Entity
- Weak Entity



1. Strong Entity

A **Strong Entity** is an entity that can be uniquely identified using its own attributes.

- It has a key attribute (Primary Key).
- It does not depend on any other entity for identification.
- Each instance of a strong entity can be identified independently.

Example:

- Student (StudentID)
- Employee (EmployeeID)
- Product (ProductID)

Here, the ID itself uniquely identifies each record.

Representation in ER Diagram:

Shown using a single rectangle.

2. Weak Entity

A **Weak Entity** cannot be uniquely identified by its own attributes alone.

- It depends on a strong entity for identification.
- It does not have a full primary key of its own.
- It uses a partial key along with the strong entity's key.
- A weak entity is connected to a strong (identifying) entity, which helps to identify it.

Example:

- Dependent (depends on Employee)
- Order Item (depends on Order)
- Room (depends on Hotel)

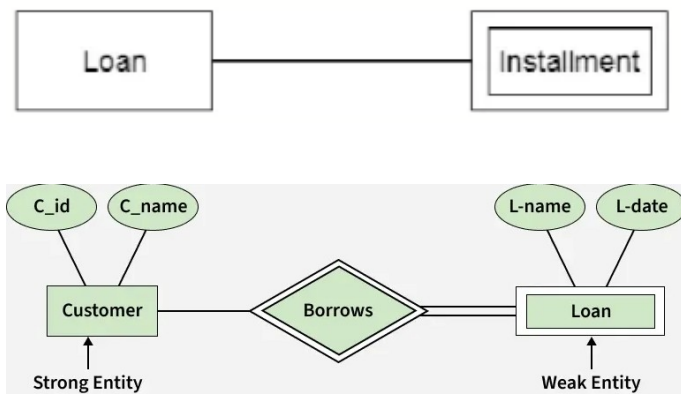
A “Room” number alone may not be unique, but Room No + Hotel ID makes it unique.

Strong Entity



Weak Entity

Example: A company may store the information of dependents (Parents, Children, Spouse) of an Employee. But the dependents cannot exist without the employee. So **Dependent** will be a **Weak Entity Type** and **Employee** will be the identifying entity type for Dependent, which means it is a **Strong Entity Type**.



11.2. Attribute :

An attribute is a property or characteristic that describes an entity. Attributes store the actual data.

Examples :

For the entity **Student**:

- StudentID
- Name
- Age
- Email

For the entity **Course**:

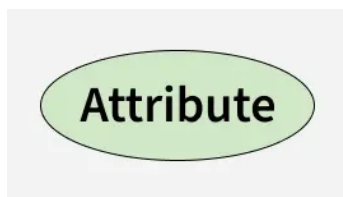
- CourseID
- CourseName
- Duration

Attributes become columns in a table.

Examples:

- Student → StudentID, Name, Age
- Course → CourseID, CourseName, Credits
- Employee → EmployeeID, EmployeeEmail

In an ER diagram, an attribute is shown using an oval shape



11.2.1. Types of Attributes

There are four main types of attributes in the ER Model:

1. Key Attribute

A **Key Attribute** is an attribute that uniquely identifies each entity in an entity set.

- Its value is different for every record.
- It is used to distinguish one entity from another.

Example: Roll_No for Student (each student has a unique roll number)

ER Diagram Representation: Shown as an oval with an underline.

2. Composite Attribute

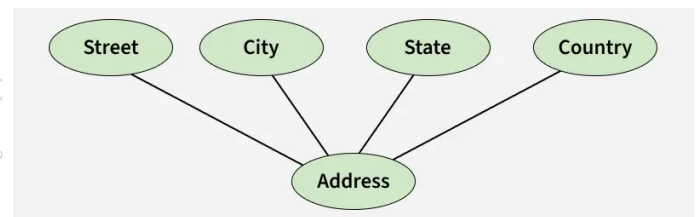
A **Composite Attribute** is an attribute that can be divided into smaller sub-parts.

Example: Address can be divided into:

- Street
- City
- State
- Country

So, Address is a composite attribute made of smaller attributes.

ER Diagram Representation: Shown as a main oval connected to smaller ovals.



3. Multivalued Attribute

A **Multivalued Attribute** is an attribute that can have more than one value for a single entity.

Example: Phone_No (a student may have multiple phone numbers)

ER Diagram Representation: Shown using a double oval.

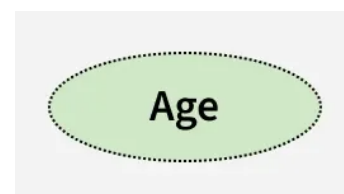


4. Derived Attribute

A **Derived Attribute** is an attribute whose value can be calculated from other attributes.

Example: Age can be calculated from DOB (Date of Birth)

ER Diagram Representation: Shown as a dashed oval.



The Complete Entity Type Student with its Attributes can be represented as:

Relationship : A relationship represents how two or more entities are connected to each other.

Examples:

- Student enrolls in Course
- Employee works for Company

Relationships show how data in different tables is linked.

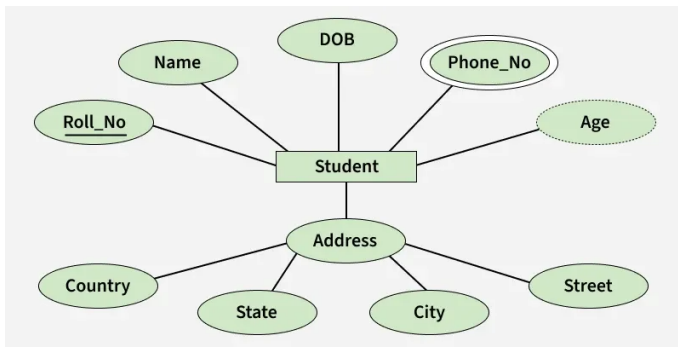


Figure 1. Entity and Attributes

11.3. Relationship Type and Relationship Set

Relationship Type

A **Relationship Type** shows how two or more entity types are connected. It describes the kind of association between entities.

Example: Student *Enrolled in* Course

Here, “Enrolled in” is the relationship type between Student and Course.

ER Diagram Representation: A relationship type is shown using a **diamond shape**, connected to entities with lines.

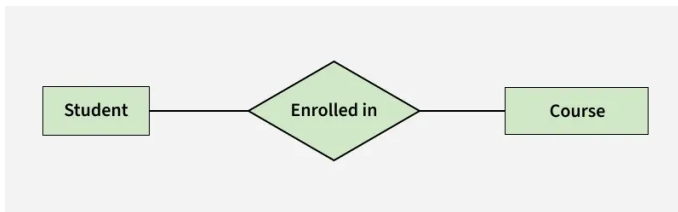


Figure 2. Entity-Relationship Set

Relationship Set

A **Relationship Set** is the collection of all relationships of the same type. It shows the actual connections between entity instances.

Example:

Student	Course
S1	C2
S2	C1
S3	C3

All these individual enrollments together form the relationship set for “Enrolled in”.

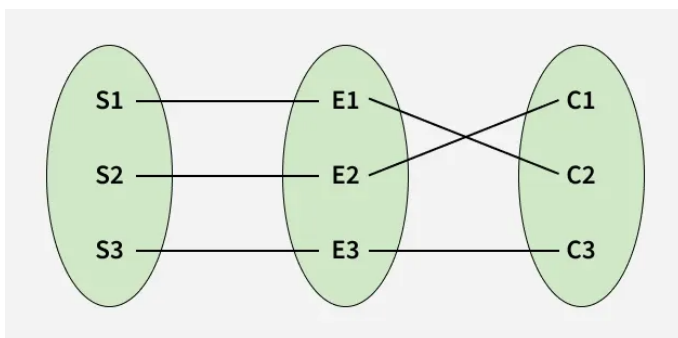


Figure 3. Relationship Set

Degree of a Relationship Set

The **degree of a relationship set** means the number of entity sets involved in that relationship.

1. Unary (Recursive) Relationship

When only **one entity set** participates in a relationship. An entity is related to itself.

Examples:

- A person is married to another person
- An employee manages another employee

Here, the same entity type participates more than once.

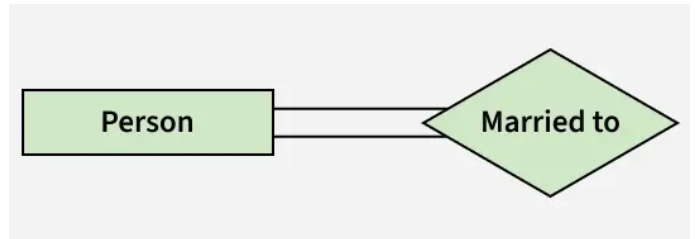


Figure 4. Unary Relationship

2. Binary Relationship

When **two entity sets** participate in a relationship. This is the most common type.

Examples:

- Student enrolled in Course
- Customer places Order

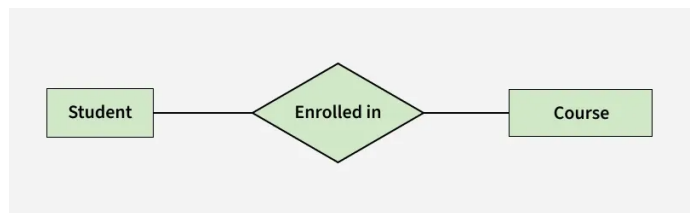


Figure 5. Binary Relationship

3. Ternary Relationship

When **three entity sets** participate in a relationship. (Relationship among three entity sets.)

Example: Supplier supplies Product to Store

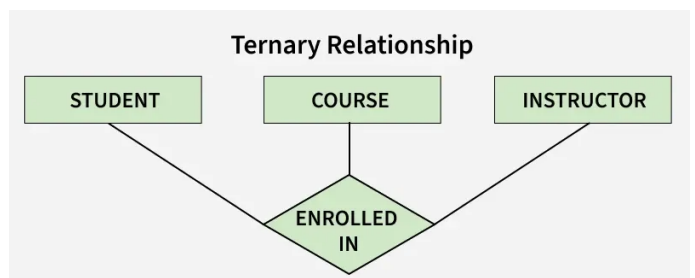


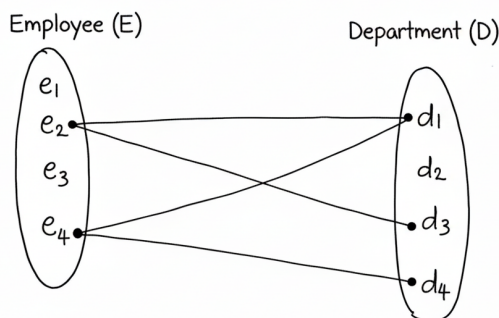
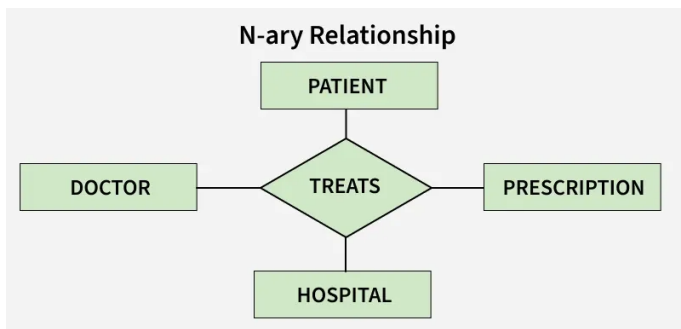
Figure 6. Ternary Relationship

4. N-ary Relationship

When more than three (n) entity sets participate in a relationship. A Relationship among n -entity set.

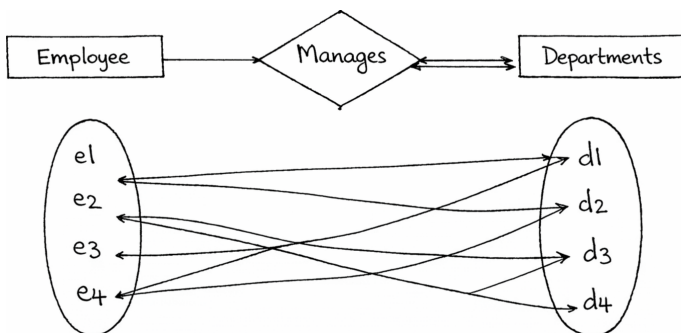
11.4. Key Constraint :

An entity is acting as a key to another entity through the relationship set it's denoted in E-R Model using an Arrow. "Each department is managed by at most one employee".



11.5. Participation Constraint :

If every entity in the entityset participates in a relationship set is called total participation denoted by double line(thick line). Otherwise it is partial participation.(thin line or single line). "Each department is managed by atleast one employee".



Cardinality in ER Model

Cardinality tells us the maximum number of times an entity from one entity set can participate in a relationship with entities of another set. "How many of one thing can be related to another?"

Types of Cardinality

11.5.1. 1. One-to-One (1:1)

In a one-to-one relationship, each entity in both sets is related to only one entity in the other set.

Example:

- One person can have only one passport
- One passport belongs to only one person

So, Person ↔ Passport = 1:1

- Each person has one passport
- Each passport belongs to one person

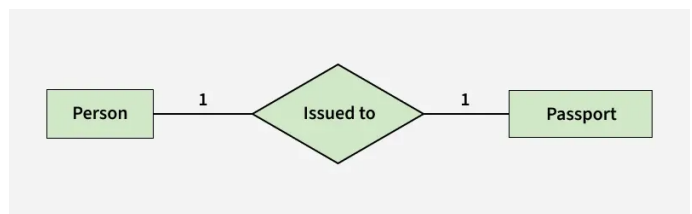


Figure 7. one to one cardinality

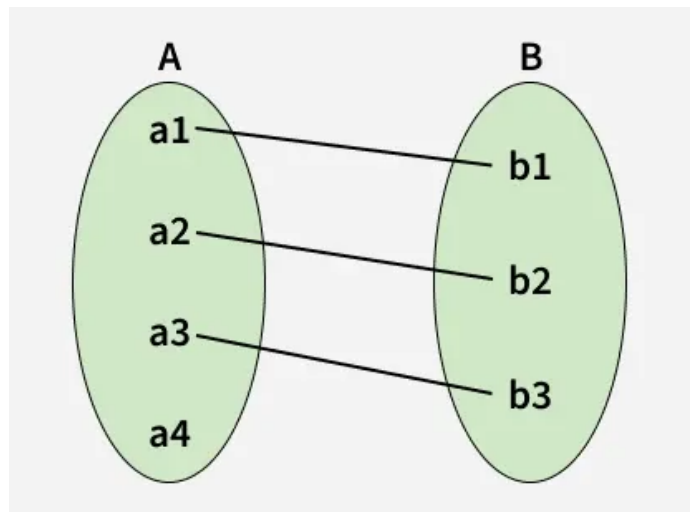


Figure 8. Set Representation of One-to-One

11.5.2. 2. One-to-Many (1:M)

In a one-to-many relationship, one entity from the first set can be related to many entities in the second set, but each entity in the second set relates to only one in the first set.

Example:

- One Department has many Doctors
- Each Doctor belongs to only one Department

So, Department → Doctors = 1:M

- One department → many doctors
- One doctor → only one department

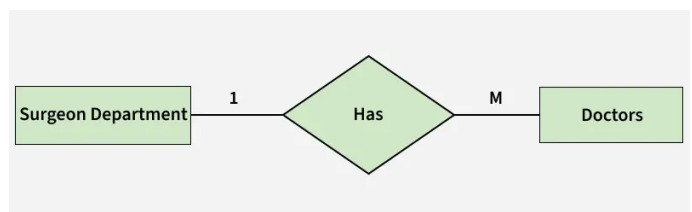


Figure 9. one to many cardinality

11.5.3. 3. Many-to-One (M:1)

This is the reverse of one-to-many. Many entities from the first set are related to one entity in the second set.

Example:

- Many surgeries are performed by one surgeon
- Each surgery is done by only one surgeon

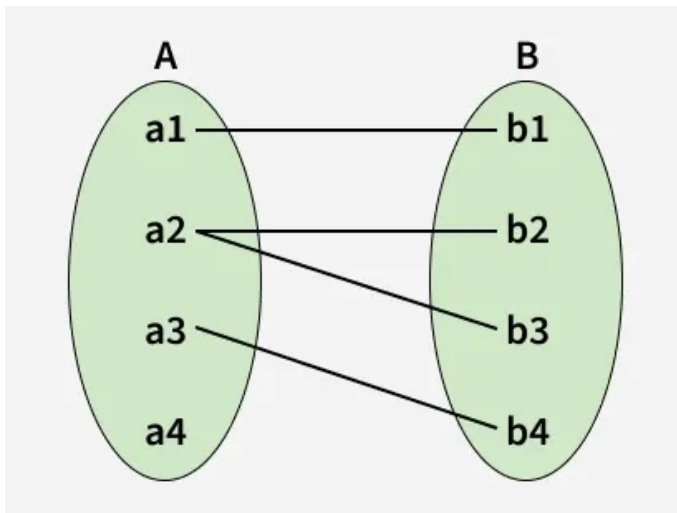


Figure 10. Set Representation of One-to-One

So, Surgeries → Surgeon = M:1

Another Example:

- Many students take one course
- Each student is taking only one course

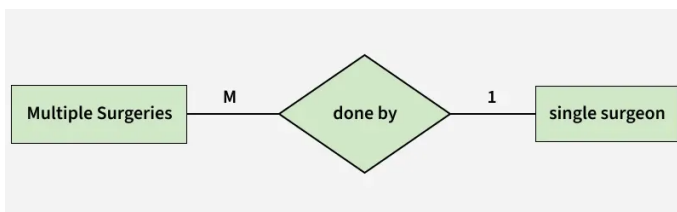


Figure 11. many to one cardinality

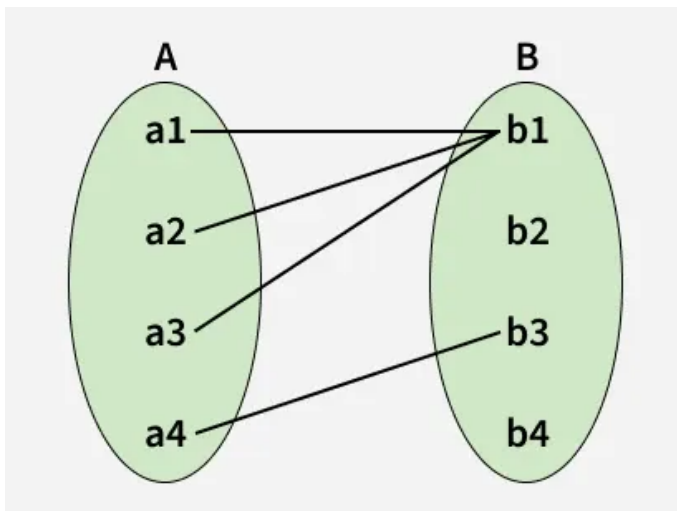


Figure 12. Set Representation of Many-to-One

11.5.4. 4. Many-to-Many (M:N)

In a many-to-many relationship, entities in both sets can be related to multiple entities in the other set.

Example:

- An employee can work on many projects
- A project can have many employees

So, Employee ↔ Project = M:N

Another Example:

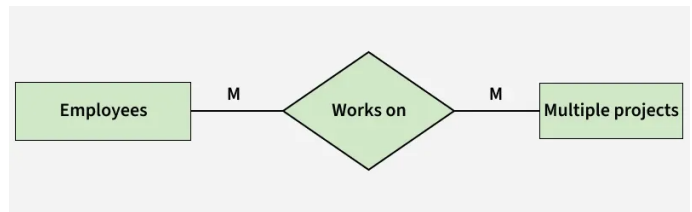


Figure 13. many to many cardinality

- Student S1 enrolls in C1 and C3
- Course C3 has S1, S3, and S4

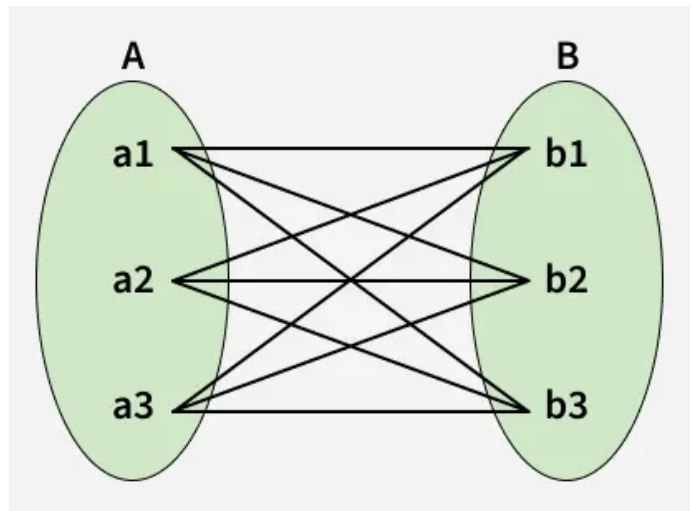


Figure 14. Many-to-Many Set Representation

Steps to Design an ER Diagram

(a) Identify Entities

The first step is to identify all the entities involved in the system. Entities are real-world objects or concepts such as *Student*, *Course*, *Employee*, or *Product*. Each entity is represented by a rectangle with its name written inside.

(b) Identify Relationships

Next, determine how the entities are related to each other. A relationship shows the connection between entities. It is represented by a diamond shape and connected to the related entities using lines. Relationships should not be directly connected to other relationships.

(c) Add Attributes

After defining entities and relationships, add attributes. Attributes describe the properties of entities, such as *Name*, *Age*, or *Roll Number*. They are shown using oval shapes and connected to their respective entity. One entity can have multiple attributes.

(d) Define Primary Keys

Each entity must have a primary key, which uniquely identifies every record. For example, *Roll Number* for *Student*. Primary keys are shown as underlined attributes in the ER diagram.

(e) Remove Redundancies

Review the diagram and remove repeated entities, unnecessary relationships, and duplicate attributes. This keeps the design clean and efficient.

(f) Review for Clarity

Finally, check the diagram to make sure it is clear and easy to understand. It should properly show how entities, attributes, and relationships are connected.